循环吧，小循环！

Zitian Gao **Yilong Chen** **Yihao Xiao** **Xinyu Yang**

Ran Tao **Joey Zhou** **Bryan Dai**

IQuest Research

在此查看完整的作者贡献。

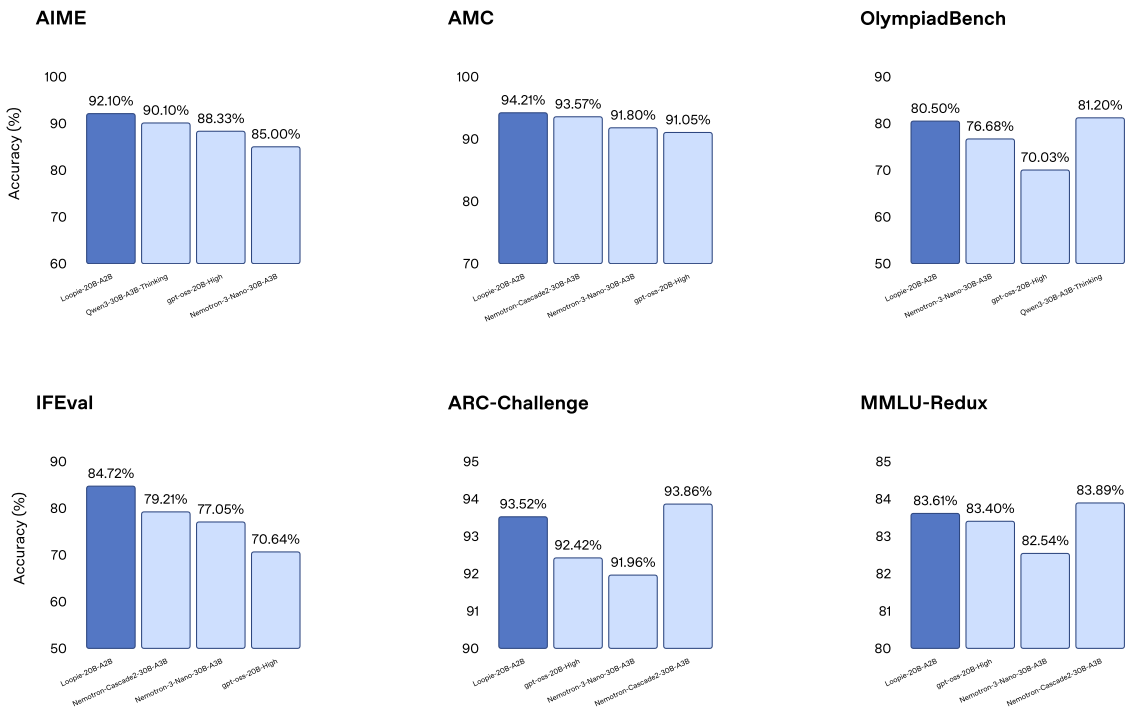
🤗 **模型：** Loopie-20B-A2B Loopie-6B-A0.6B

代码： megatron-loopie vllm-loopie

arXiv:2607.16051v2 [cs.CL] 20 Jul 2026

Abstract

本文提出 Loopie 系列，包含两个专家混合（Mixture-of-Experts，MoE）模型：一个具有 20B 参数、2B 活跃参数的模型，以及一个具有 6B 参数、0.6B 活跃参数的模型。循环变换器（Looped Transformer）长期面临一个挑战：在预训练计算量增加 N 倍的情况下，将参数数量增加 N 倍通常优于将模型循环 N 次。Loopie 解决了这一挑战。广泛的消融研究（包括与普通 30B-A3B 模型的比较）表明，Loopie 显著优于在相同计算预算下训练的普通变换器基线。通过一种新颖的后训练方法，Loopie 发展出强大的推理能力，并达到了前沿水平的推理性能。



目录

1 引言 3

2 Loopie 系列 4 2.1 架构 . . . . . 4 2.2 为何采用层循环? . . . . . 5 2.3 动机 . . . . . 6 2.4 Loopie 配方 . . . . . 6 2.5 结果 . . . . . 9 2.6 可扩展性 . . . . . 10 2.7 消融 . . . . . 11 2.8 为何仅两个循环步骤? 11

3 预训练 13 3.1 评估 . . . . . 13 3.2 初始化 . . . . . 13 3.3 学习计划 . . . . . 13 3.4 阶段 1：多轮次高质量预训练 . . . . . 14 3.5 阶段 2：高质量退火 14

4 后训练 15 4.1 监督式预训练 . . . . . 16 4.2 强化学习 . . . . . 18 4.3 结果 19

5 相关工作 21 5.1 循环变换器 . . . . . 21 5.2 归纳偏置 . . . . . 22 5.3 理论、机制与扩展 . . . . . 22 5.4 架构、训练目标与推理 . . . . . 23 5.5 上下文、算法与组合泛化 . . . . . 24 5.6 抽象推理 24

6 未来工作 24

7 结论 25

8 致谢 26

附录 A 架构细节 . . . . . 35 B 预训练细节 . . . . . 36 C 缩放阶梯细节 . . . . . 37 D 监督预训练细节 38

1 引言

循环变换器（Looped Transformers）最初以通用变换器（Universal Transformers）之名提出（Dehghani 等，2019），近期重新成为传统深度扩展的一种有力替代方案。与堆叠不同层不同，它们在循环步骤中反复应用同一模型，使该方法与变换器中的参数共享紧密相关（Lan 等，2020；Dehghani 等，2019）。这种循环方法在语言建模、算法学习和抽象推理等广泛领域展现出强大的经验性能（Gao 等，2025b；Yang 等，2024；Jolicoeur-Martineau，2025；Wang 等，2025；Dehghani 等，2019；Saunshi 等，2025；Bae 等，2025；Geiping 等，2025；Zhu 等，2025；Frey 等，2026a；Huang 等，2026b）。越来越多的研究表明，循环计算特别适合复杂问题。循环变换器在上下文学习和数据拟合任务上可超越普通变换器，并能以少得多的参数在上下文中实现多步梯度下降（Yang 等，2024；Fan 等，2025；Giannou 等，2023；Gatmiry 等，2024a,b；Chen 等，2025）。对循环变换器的研究还表明，循环计算使浅层参数化在推理任务上接近更深层非绑定模型的性能（Saunshi 等，2025；Jeddi 等，2026；Geiping 等，2025；Zhu 等，2025）。这些优势在抽象推理任务（如 ARC-AGI、数独和迷宫）上尤为明显，近期循环模型的结果表明，循环为组合泛化、规则发现和归纳推理提供了强大的归纳偏置（Wang 等，2025；Jolicoeur-Martineau，2025；Gao 等，2025b）。

近期工作已开始将循环变换器（Looped Transformer）扩展到十亿参数级语言模型。Ouro 模型通过四个循环步骤扩展到 14 亿和 26 亿参数，相对于更大的密集基线模型，报告了显著的参数效率（Zhu 等，2025）。Huginn 扩展到 35 亿参数，并使用 32 个循环步骤以参数换取潜在计算（Geiping 等，2025）。其他系统进一步研究了自适应循环、高效推理、内存-计算权衡以及稳定循环语言模型的缩放定律（Bae 等，2025；Jeddi 等，2026；Frey 等，2026a；Prairie 等，2026）。然而，这些收益暴露了一个根本性的计算核算问题：在预训练期间将模型循环 N 次也会使预训练计算量乘以 N。因此，循环变换器不仅应与具有相同参数量的普通变换器进行比较，还应与在相同预训练计算预算下训练的非循环模型进行比较。这一观点与语言模型缩放和计算最优训练分析一致，后者将质量评估为参数、数据和总训练浮点运算次数的函数，而不仅仅是参数数量（Kaplan 等，2020；Hoffmann 等，2022；Prairie 等，2026）。例如，具有 4 个循环的 Ouro-2.6B 应与参数量接近 2.6B $\times$ 4 = 10.4B 的基线模型进行比较；同样，在考虑预训练计算后，具有 32 个循环的 Huginn-3.5B 应与一个更大的 112B 参数基线模型进行比较。这引出了我们的核心问题：

> 在相同的预训练计算预算下，循环变换器能否达到或超过普通变换器？

我们通过 Loopie 系列来回答这个问题：两个循环混合专家（MoE）大语言模型，Loopie-20B-A2B 和 Loopie-6B-A0.6B，每个模型都使用两个循环步骤进行训练。我们的关键思想是 Loopie 配方，这是一种计算匹配的缩放配方，解决了循环深度缩放的主要问题：在固定的预训练计算预算下，普通参数缩放可能会主导循环（Hoffmann 等，2022；Prairie 等，2026）。通过一种新颖的后训练方法，Loopie 发展了强大的推理能力，并达到了前沿水平的推理性能。

我们的贡献有三方面：

- **计算匹配的缩放。**我们引入了 Loopie 配方来解决循环变换器在固定计算预算下的挑战，并通过广泛的消融实验验证了它。
- **可扩展的循环混合专家模型。**我们通过训练 Loopie-20B-A2B 和 Loopie-6B-A0.6B，证明了循环计算可扩展到大型 MoE 语言模型。
- **大规模后训练。**我们通过大规模后训练来扩展 Loopie，包括一个新颖的监督预训练阶段，从而获得强大的推理能力。

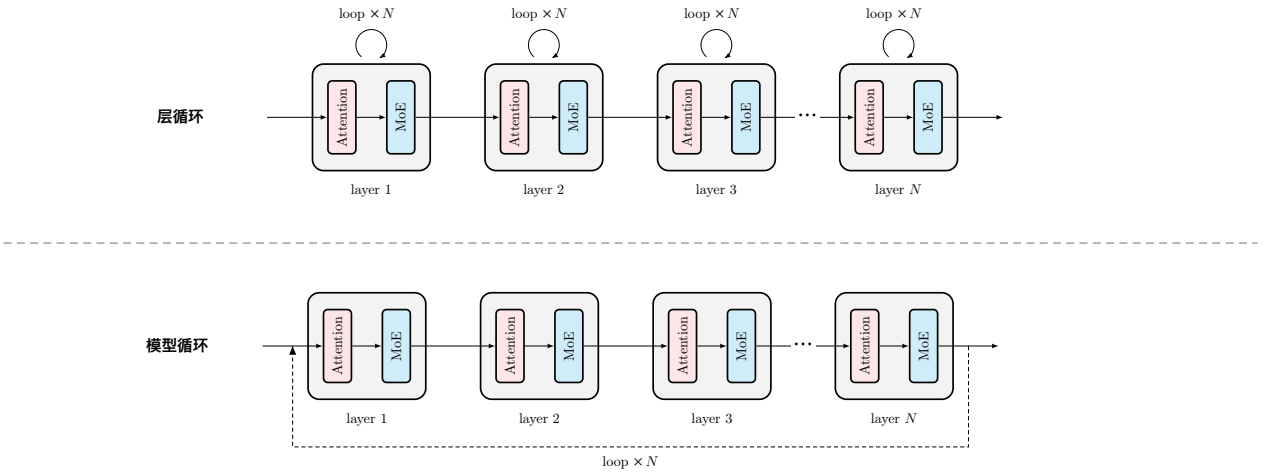


图 1 层循环与模型循环对比示意图。上图展示层循环，即 Loopie 采用的循环调度，其中每个注意力/MoE 层在将其输出传递给下一层之前被循环应用。我们证明该调度在 MoE 骨干上取得更好性能，同时训练更高效。下图展示传统的整体模型循环模式，我们称之为模型循环，即遍历整个层堆栈后重复。该模式被先前的循环模型如 Ouro 和 Huginn 所采用（Zhu et al., 2025; Geiping et al., 2025）。

2 Loopie 系列

本节介绍 Loopie 系列，从其架构到可扩展性证据。我们首先定义 Loopie 的层循环模式，并将其与先前循环语言模型中使用的经典模型循环模式进行对比。然后在固定预训练计算预算下阐述设计动机，其中使用循环深度的模型必须与在普通非循环容量上花费相同计算量的普通 Transformer 进行比较。基于这一动机，我们引入 Loopie 配方，一种计算匹配的扩展配方，用于选择存储宽度、存储深度和循环深度。接下来我们展示主要的计算匹配结果，检验该优势是否跨模型规模持续存在，并通过消融实验分离层循环模式的贡献。最后，我们讨论为什么 Loopie 仅使用两个循环步骤。

2.1 架构

Loopie 大体上遵循 Qwen3-MoE 架构。具体而言，其主干是一个仅解码器的专家混合变换器，其注意力机制、稀疏模式及其他架构细节与 Qwen3-MoE 系列保持一致。详细的架构规格见附录 A。关键的架构差异在于循环计算的应用方式。Loopie 并非简单地将整个模型重复多次，而是采用了一种不同的循环模式，我们称之为层循环。先前的循环语言模型，如 Ouro 和 Huginn（Zhu 等，2025；Geiping 等，2025），主要采用我们所说的模型循环。在模型循环中，整个变换器堆栈被循环展开：对于一个具有三层和两个循环步的模型，计算顺序为

$$\text{第 1 层} \to \text{第 2 层} \to \text{第 3 层} \to \text{第 1 层} \to \text{第 2 层} \to \text{第 3 层}.$$

相比之下，Loopie 使用层循环，即每一层在计算进入下一层之前被循环应用。对于同样的三层、两步示例，计算顺序变为

$$\text{第 1 层} \to \text{第 1 层} \to \text{第 2 层} \to \text{第 2 层} \to \text{第 3 层} \to \text{第 3 层}.$$

也就是说，每个块对隐藏状态执行局部迭代，然后才将得到的循环表示传递给下一层。图 1 展示了这一差异。

2.2 为什么采用层循环?

层循环不仅仅是循环计算顺序的不同；它改变了模型内部迭代细化的位置，因此影响扩展行为、执行效率以及跨有效深度的参数共享性质。我们采用层循环是因为它提供了三个优势，这些优势对于大规模预训练尤为重要。

更好的经验扩展性。
在我们的预训练实验中，层循环比模型循环取得了更好的性能。如图 2 所示，层循环在下游基准上最初略落后于模型循环。然而，它在大约 1.2 万亿训练标记后超过了模型循环，并且此后提升更快。因此，随着训练预算的增加，模型循环的早期优势不会持续。

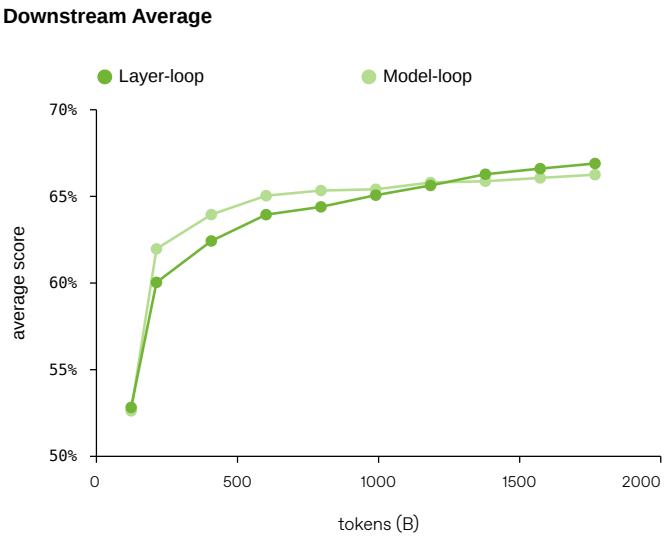


图 2 Loopie-6B-A0.6B 的层循环和模型循环变体之间平均下游基准分数的比较。尽管层循环在训练早期落后于模型循环，但它在后期超过了模型循环。

基础设施友好的执行。尽管层循环和模型循环具有相同的标称层应用次数和理论浮点运算量，但层循环提供了更好的执行局部性。同一层的重复应用在前向和后向计算图中是相邻的，这缩短了参数和梯度贡献的重用距离，并简化了激活检查点和梯度累积，尤其是在参数分片或卸载的情况下。这种局部性对流水线并行特别有利，因为层的所有循环应用都保留在同一流水线阶段内，然后激活才会向前传递。相比之下，模型循环要求每个微批次反复遍历整个流水线，并在每个循环边界将最后阶段的输出路由回第一阶段，引入了循环依赖，这可能使调度复杂化，增加通信开销和流水线气泡的数量，并降低设备利用率。

自然的参数共享模式。例如，考虑一个具有两个模型循环步骤的 48 层模型，如 Qwen3-30B-A3B。第三个物理层在有效深度 3 和 $48 + 3 = 51$ 处被应用。这两次调用发生在明显不同的先前计算量之后，因此在相距甚远的有效深度处接收隐藏状态。先前的分析表明，变换器表示在深度上是不均匀组织的：较低层和最终层可能与相对同质的中间层有显著差异，而不同的语言抽象往往在网络的不同阶段最容易获得（Sun 等人，2025；Tenney 等人，2019；Jawahar 等人，2019）。这些观察结果并不

直接建立梯度冲突，但它们表明，在相隔较远的有效深度上，隐藏状态无需对共享变换提出完全相同的功能需求。因此，模型循环要求单一参数集适应可能异质的深度相关角色。相比之下，层循环在相邻的有效深度上重用某一层，从而形成更局部且可能更连贯的参数共享模式。

2.3 动机

尽管循环变换器在概念上具有吸引力，但历史上对其的研究环境并未充分反映现代大规模语言模型预训练的实际约束。具体而言，先前的工作大多聚焦于稠密、权重共享或循环变换器变体，并主要将循环机制评估为提升参数效率的手段（Zhu 等，2025；Geiping 等，2025）。这导致两个重要问题尚未得到充分探索。首先，现代前沿语言模型日益依赖专家混合架构，近期发布的 Qwen3、Kimi K2.5、GLM-5、DeepSeek-V4 和 MiniMax-M2 模型系列即为典型例证（Qwen 团队，2025；Kimi 团队等，2026；GLM-5 团队等，2026；DeepSeek-AI，2026；MiniMax 等，2026）。专家混合模型在扩大总模型容量的同时，保持每个词元的活跃参数数量相对较少，使其在训练和推理计算约束下显著优于稠密模型（Shazeer 等，2017；Fedus 等，2022；Du 等，2022；Abnar 等，2025）。因此，实用的循环架构应当兼容专家混合扩展，而非局限于稠密骨干网络，因为这种限制将制约其扩展到更大模型规模的能力。其次，预训练计算通常是大模型训练中的主导约束。它决定了模型规模的上限、可处理的词元数量以及可训练的实验变体数量（Kaplan 等，2020；Hoffmann 等，2022）。这为循环变换器带来了一个关键难题：循环计算并非零成本。若模型在预训练期间循环 N 次，其训练计算量也将近似乘以 N。因此，循环模型不应仅仅优于具有相同存储参数量的普通变换器。为了证明循环机制作为一种扩展策略的合理性，它必须与在相同预训练计算预算下训练的普通变换器相竞争。这一要求颇具挑战性。先前的研究表明，在固定浮点运算次数的比较下，循环模型的非循环容量可能低于标准变换器基线（Csordás 等，2024；Frey 等，2026b）。这与传统观点一致，即循环机制主要提升参数效率，但未必提升计算效率（Lan 等，2020；Takase 和 Kiyono，2023）。因此，先前的循环变换器尚未为在万亿参数规模上实现前沿性能的旗舰模型提供清晰路径。Loopie 旨在填补这一空白。我们的核心观察是，只有在搭配适当的循环宽度-深度权衡时，循环机制才能在固定计算预算下具备竞争力。我们将这一原则形式化为 Loopie 配方：不应将循环视为额外参数的直接替代品，而应联合选择存储宽度、存储深度和循环深度，使所得模型在固定预训练计算预算下最大化性能。这将循环机制的角色从节省参数的装置转变为计算匹配的扩展机制。

2.4 Loopie 配方

本文现描述用于实例化 Loopie-20B-A2B 的经验性、硬件感知的扩展流程。从一个强大的非循环混合专家（MoE）参考模型出发，Loopie 配方包含三个步骤：

（i）通过将存储层数减半来构建循环种子模型；
（ii）利用层循环（layer-loop）将每个存储层执行两次；以及
（iii）利用由此产生的内存余量将每设备微批次大小加倍，然后将实测的训练效率增益再投入到额外的模型容量中，同时保持优化器步长时间与参考模型大致匹配。

本文通过实际预训练成本而非精确的理论浮点运算次数（FLOPs）来匹配模型。对于

每次比较，本文固定硬件分配、序列长度、每个优化器步的令牌数、激活检查点策略、优化器和训练数据。不同架构可根据其实测内存占用使用不同的每设备微批次大小。当微批次大小增加时，梯度累积步数按比例减少，以使每个优化器步的令牌数保持不变。本文选择实测端到端优化器步长时间最接近非循环参考模型的 Loopie 配置。由于两个模型使用相同的令牌预算和优化器更新次数，这也大致匹配了总墙钟训练成本。模型不按理论 FLOPs 匹配。Loopie 每个令牌执行更多的名义计算，但额外的工作被更高的实际效率所抵消。本文将这种操作称为计算匹配比较。分析性 FLOP 和内存模型仅用于构建和解释候选配置。最终架构根据 Megatron-LM（Shoeybi 等人，2019）中实测的端到端训练时间进行选择。实测效率增益完全来自存储深度的减少，这降低了激活内存，使本文能够将每设备微批次大小加倍，同时将梯度累积步数减半。

激活内存与微批次效率。在我们实验所用的检查点实现下，存储层的所有循环应用都被包含在同一个检查点单元中。因此，占主导地位的激活内存项随存储深度而非执行深度扩展。训练期间的激活内存 M_{act} 按以下方式扩展（Korthikanti 等人，2023；Rajbhandari 等人，2020）

$$M_{\text{act}} \propto s\,b\,D\,L,$$

其中 D 是隐藏维度，L 是存储层数，b 是每设备微批大小，s 是序列长度。循环应用通过循环计数 R 增加了执行的计算量，但在这种检查点方案下不会引入 R 独立存储的层边界激活集合。

对于固定的全局批大小 B，每设备微批大小为 b，梯度累积步数为 g，我们有

$$g = \frac{B}{b}.$$

减少 DL 可腾出内存空间，用于增加 b。由于梯度累积步数 g 按比例减少，全局批大小保持不变：

$$b_1 = 2b_0, \qquad g_1 = \frac{g_0}{2}.$$

更大的微批为每个核函数暴露了更多并行工作，并减少了每次优化器更新所需的梯度累积微步数。对于固定架构，相应的微批效率增益直接测量为

$$S_{\text{mb}} = \frac{t_{\text{step}}(D, L, R; b, g)}{t_{\text{step}}(D, L, R; 2b, g/2)}.$$

这个测量量就是 Loopie 配方所使用的效率因子。我们使用类似 Qwen3 的 30B-A3B 专家混合变换器（Qwen 团队，2025）作为非循环参考，其中

$$D_0 = 2048, \qquad L_0 = 48, \qquad R_0 = 1.$$

第一步是通过将存储层数减半并设置 $R = 2$ 来构建循环种子模型：

$$D = 2048, \qquad L = 24, \qquad R = 2.$$

主导阶预训练计算（Hoffmann 等人，2022）按以下方式扩展

$$C(D, L, R) \propto L D^2 R.$$

表 1 由类 Qwen3 的 30B-A3B 参考模型生成的候选配置。 $\widehat{C}$ 是归一化的前导阶变换器块工作代理，$\widehat{M}_{\text{act}}$ 是在固定每设备微批次大小下的归一化激活内存代理。最终计算匹配基于实测的优化器步长时间。

Configuration	D	L	R	$\widehat{C}$	$\widehat{M}_{\text{act}}$
Qwen3 30B-A3B	2048	48	1	1.00×	1.00×
Seed Loopie	2048	24	2	1.00×	0.50×
Loopie candidate 1	2176	25	2	1.18×	0.55×
Loopie candidate 2	2304	27	2	1.42×	0.63×
Loopie candidate 3	2432	28	2	1.65×	0.69×

在固定宽度下，该变换保持变换器块执行次数不变：

$$LR = 24 \cdot 2 = 48 = L_0 R_0,$$

因此保持前导阶变换器块计算代理不变：

$$LRD^2 = 24 \cdot 2 \cdot 2048^2 = 48 \cdot 2048^2.$$

在相同的每设备微批次大小下，主导激活内存项减半：

$$\frac{DL}{D_0 L_0} = \frac{2048 \cdot 24}{2048 \cdot 48} = 0.5.$$

因此，循环种子模型执行的前导阶变换器块工作与参考模型大致相同，同时存储激活所需的内存显著减少（Shoeybi 等，2019；Rajbhandari 等，2020）。第二步是将由此产生的微批次效率用于增加模型容量。为满足架构和硬件对齐约束，我们将候选隐藏大小限制为 128 的倍数。我们在循环种子模型周围扫描可行的 (D, L) 配置，并保留支持两倍于参考模型微批次大小的候选配置。表 1 列出了候选配置。我们报告归一化的前导阶计算代理：

$$\widehat{C} = \frac{LRD^2}{48 \cdot 2048^2},$$

以及在参考微批次大小下的激活内存代理，

$$\widehat{M}_{\text{act}} = \frac{DL}{48 \cdot 2048}.$$

这些量用于描述候选配置，而非预测其最终优化器步长时间。我们在 Megatron-LM 中对每个候选配置进行大规模基准测试，遵循上述匹配协议。对于参考模型和每个候选模型，我们联合搜索张量并行、专家并行和微批次大小。然后选择实测优化器步长时间最接近非循环参考模型的候选配置：

$$D_1 = 2304, \qquad L_1 = 27, \qquad R_1 = 2,$$

由此定义 Loopie-20B-A2B。在参考每设备微批次大小下，Loopie-20B-A2B 相对于参考模型的主导激活内存代理为

$$\frac{D_1 L_1}{D_0 L_0} = \frac{2304 \cdot 27}{2048 \cdot 48} \approx 0.633.$$

该表达式是一个缩放代理，而非精确的峰值内存方程。因此，候选可行性是通过实测的完整内存占用来确定的，其中还包括参数、优化器

状态、临时缓冲区与通信工作区。Loopie-20B-A2B 的实测内存分布允许将每设备微批次大小加倍：

$$b_1 = 2b_0, \qquad g_1 = \frac{g_0}{2}.$$

这使得每个优化器步骤的令牌数量保持不变。所选模型的归一化前导阶变换器块计算代理为

$$\frac{\widehat{C}_1}{\widehat{C}_0} = \frac{27 \cdot 2 \cdot 2304^2}{48 \cdot 2048^2} \approx 1.424.$$

Loopie-20B-A2B 仅在将调度从 (b_0, g_0) 改为 $(2b_0, g_0/2)$ 后，才达到参考优化器步长时间；因此，所测得的效率提升完全归因于微批次大小翻倍。因此，计算匹配准则即为直接测得的关系：

$$t_{\text{step}}\left(D_1, L_1, R_1; 2b_0, g_0/2\right) \approx t_{\text{step}}\left(D_0, L_0, R_0; b_0, g_0\right),$$

而非基于 $\widehat{C}$ 的解析预测。主导阶计算代理忽略了低阶算子、路由与通信开销、优化器开销、核函数启动开销，以及微批次调度对实际硬件利用率的影响。因此，该代理用于刻画名义工作量，而非替代端到端计时。我们在多个图形处理器（GPU）平台上评估了 Loopie 配方，并观察到一致的基础设施级收益。

因此，Loopie 配方并不声称在分析性浮点运算次数（FLOP）估计上达到相等。它利用层循环递归来减少存储的内存占用，将由此产生的内存余量转化为每设备微批次大小的翻倍，将实测的微批次效率增益用于增加模型容量，并根据实测的端到端优化器步长时间来选择最终架构。

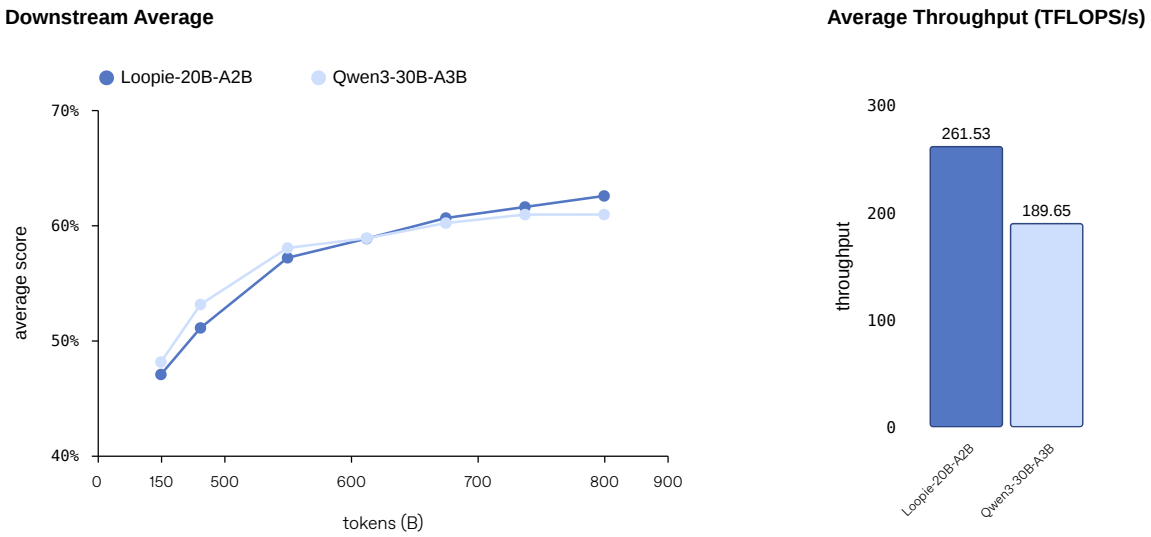


图 3 在匹配的单步预训练墙钟时间下，Loopie-20B-A2B 与本文复现的 Qwen3-30B-A3B 在 Megatron-LM 中的比较。左图报告八个下游基准的平均得分，右图报告大规模基准测试中各模型达到的最高平均预训练吞吐量。右图中，Loopie-20B-A2B 使用 EP = 8 和 MBS = 2，而 Qwen3-30B-A3B 使用 EP = 8 和 MBS = 1。这些配置在大规模 TP/PP/EP/MBS 网格搜索中达到最高吞吐量；最优设置可能因 GPU 不同而异。

2.5 结果

为验证这一原则，本文将环路模型（Loopie）与基于通义千问三混合专家（Qwen3-MoE）设计的强非循环基线进行对比。具体而言，本文在 800 规模的数据上训练了一个具有类通义千问三架构的普通 300 亿激活 30 亿参数的混合专家变换器（MoE Transformer）

在用于 Loopie-20B-A2B 的相同预训练计算预算下，训练了十亿个标记。如图 3 所示，Loopie 在训练早期阶段最初落后于较大的普通基线。然而，在预训练大约 6000 亿个标记后，Loopie-20B-A2B 超越了计算匹配的基线，并在此后保持了一致的优势。这一结果表明，通过正确的扩展配方，循环混合专家模型可以通过使用循环计算，而不是将相同的计算分配给更大的普通变换器，来实现更强的最终性能。

2.6 可扩展性

一种实用的循环架构不仅应优于单一计算匹配的基线，还应在模型扩展时保持有效性。许多架构思想在小规模上显示出有前景的结果，但在更大规模上因各种原因而失败。因此，本文评估了 Loopie 设计在一系列逐渐增大的预训练运行中是否保持其优势。

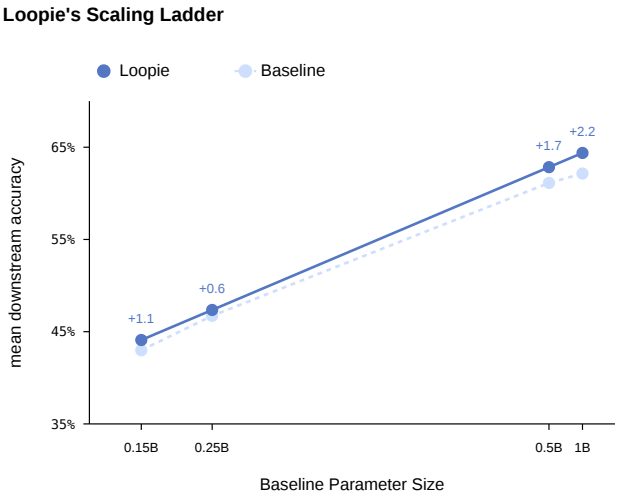


图 4 Loopie 的扩展阶梯。横轴表示非循环基线的参数量，纵轴报告八个下游基准的平均得分。Loopie 模型并未按横轴所示的参数量绘制；相反，每个 Loopie 模型根据 Loopie 配方调整规模，使其每步预训练的实际耗时与对应基线完全一致。

我们构建了一个由四个非循环混合专家基线模型和四个计算量匹配的 Loopie 模型组成的扩展阶梯。每个 Loopie 模型都是通过 Loopie 配方将非循环混合专家基线模型转换而来。在每一级阶梯上，Loopie 模型使用两个层循环步骤，其存储宽度和深度根据第 2.4 节中描述的相同计算量匹配原则进行选择。该阶梯的目标不是匹配存储参数数量，而是匹配相应普通基线模型的有效预训练计算量。因此，Loopie 模型包含更少的存储参数，同时利用循环来增加有效深度。对于每个参考模型，我们训练的令牌数量等于 1000×其活跃混合专家参数数量，确保每个模型都得到充分过度训练，并且其下游度量已基本稳定。我们对相应的 Loopie 模型使用相同的令牌预算，尽管这些模型的活跃参数更少。对于最小的阶梯级别，我们在 1500 亿个令牌上训练 0.15B 普通基线模型及其计算量匹配的 0.10B Loopie 对应模型，使这些模型相对于标准的 Chinchilla 风格计算最优方案处于严重过度训练的状态。对于 0.25B 和 0.50B 级别，我们分别在 2500 亿和 5000 亿个令牌上进行训练。对于 1B 普通基线模型和相应的 0.70B Loopie 模型，由于计算资源有限，我们也在 5000 亿个令牌上进行训练。该扩展阶梯的架构细节见附录表 7。图 4 总结了由此产生的扩展行为。在所有四个级别上，Loopie 始终优于其计算量匹配的普通对应模型。更重要的是，随着模型规模的增大，这种差距并未消失。

这表明 Loopie 的优势在多个有意义的规模阶梯上持续存在，而非仅仅是小模型工件。该结果支持 Loopie 系列的核心设计假设：在固定计算预算下，当存储宽度、存储深度和循环深度被联合选择时，循环层计算可以转化为可扩展的建模增益。综合来看，规模阶梯提供了证据，表明 Loopie 在孤立的计算匹配比较之外仍然有效。随着基线参数数量从 0.15B 增长到 1B，Loopie 模型在匹配的预训练预算下继续提供更强的下游准确率。这种行为对于大规模部署很重要：它表明层循环设计和 Loopie 配方与渐进式扩展兼容，而不是局限于狭窄的模型规模范围。

2.7 消融实验

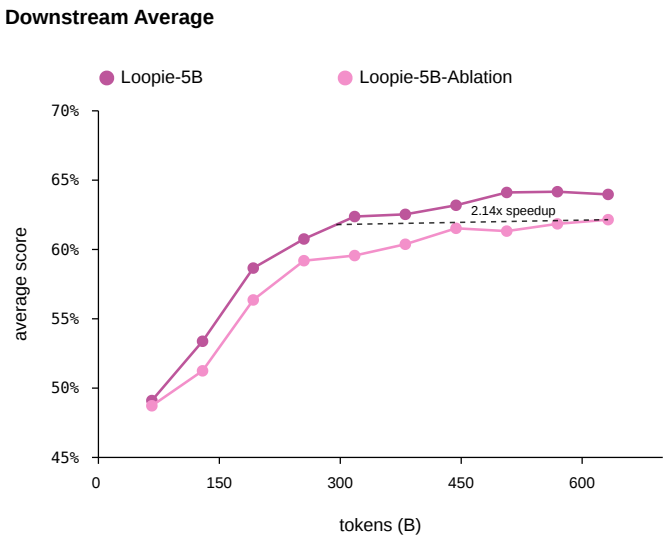


图 5 Loopie-6B-A0.6B 的层循环消融。我们报告了 Loopie-6B-A0.6B 和 Loopie-6B-A0.6B-Ablation 在八个下游基准上的平均得分。Loopie-6B-A0.6B-Ablation 在所有架构细节上与 Loopie-6B-A0.6B 匹配，除了移除了层循环递归。

除了将 Loopie 与计算匹配的普通 MoE 基线进行比较外，我们进一步隔离了层循环递归调度的贡献。为此，我们进行了一项对照实验，训练一个 6B-A0.6B MoE 模型，其骨干、优化器、数据混合和令牌预算与 Loopie-6B-A0.6B 相同，但移除了层循环模式。因此，这个消融模型保留了整体的循环计算预算，同时测试循环计算的顺序本身是否重要。图 5 显示了在八个下游基准上的平均得分。与匹配的消融相比，层循环模式显著提高了下游性能。重要的是，由于活跃参数数量和整体计算预算都没有增加，这种改进不能简单地归因于额外的计算。这一结果支持了 Loopie 中做出的架构选择。虽然循环增加了有效深度，但循环的调度方式很重要：简单地循环计算不足以实现高计算效率。这些消融表明，Loopie 的收益不仅来自使用循环，还来自以与 Transformer 表示的分层结构相一致的形式使用循环。

2.8 为何仅采用两个循环步骤?

Many recurrent language model designs use substantially deeper unrolling. For example, prior looped or latent-recurrent models often use 4, 16, or even more than 30 recurrent steps (Zhu et al., 2025; Geiping et al.,

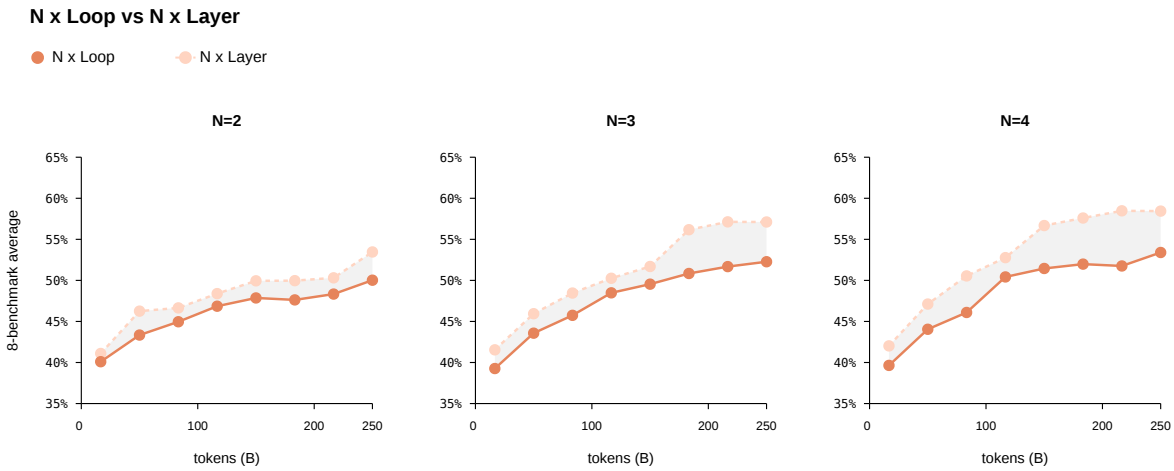


图 6 $N\times$ **层循环与** $N\times$ **存储层扩展的**比较。我们在保持所有其他架构细节不变的情况下，对每个模型在 2500 亿个词元上进行训练后，报告八个下游基准的平均得分。灰色阴影区域表示 $N\times$ 层基线与 $N\times$ 循环模型之间的差距。

2025）。这些设置对于研究循环作为迭代计算的通用机制非常有用。Loopie 针对的是不同的场景：在固定计算预算下进行大规模语言模型预训练。在此场景中，循环步骤的数量不仅仅是一个架构超参数；它直接决定了预训练浮点运算次数的分配。

在存储参数数量和优化器步骤数固定的情况下，增加循环步骤数通常会改善训练曲线。然而，这种比较并未匹配计算量。对于具有 R 个循环步骤的模型，每个词元的训练成本大致与 R 成线性关系，直至专家混合路由和实现常数：$\mathcal{C}(D,L,R) \propto LD^2R.$

在相同的预训练预算下，模型要么必须在更少的词元上进行训练，要么使用更小的存储架构，要么与一个将相同计算量用于普通变换器容量的更强的非循环模型进行比较。因此，相关的问题不在于具有 $R=4$ 个循环步骤的模型在相同存储规模下是否优于具有 $R=2$ 个循环步骤的模型，而在于额外循环带来的边际收益是否超过将相同的浮点运算次数重新分配给宽度、深度或数据所能获得的收益。

我们的循环次数扫描表明，在大规模预训练机制下，这种边际收益会迅速衰减。为了区分增益是来自循环，而非单纯增加非共享层，我们比较了一个使用 $N\times$ 层循环步骤的模型与一个存储层数多 $N\times$ 倍的模型，同时固定所有其他架构细节。图 6 报告了模型在 250B 词元上训练后，在八个下游基准上的平均得分。灰色阴影区域突出显示了 $N\times$ 层基线与 $N\times$ 循环模型之间的性能差距。对于 $2\times$ 比较，我们仅使用一个 $2\times$ 存储层模型作为先前计算匹配设置的保守代理。由于该模型非常小，仅有 0.25B 活跃参数，在多种实际约束下精确匹配计算量很困难，因此 $2\times$ 层基线的实际训练计算量显著高于 $2\times$ 循环模型。因此，该图不应被解读为 $2\times$ 循环被 $2\times$ 层扩展所支配的证据；相反，它表明，在比较 $N\times$ 循环与 $N\times$ 存储层扩展时，循环的边际收益在 $R=2$ 处最大。

因此，本文为 Loopie 系列选择 $R=2$。这是最小的非平凡循环设置：每一层执行一次普通变换，随后进行一次局部细化步骤，再将表示传递给下一层。这使得循环能够从本质上改变计算，同时保持计算乘数足够小，以支持可扩展的预训练。它还保持了训练吞吐量，并使与类似 Qwen3 的普通 MoE 基线的比较更加严格：Loopie 必须超越这些基线，而非

通过使用大量额外的循环计算，而是通过更有效地使用少量循环。这一选择不应被解读为声称更大的循环次数在孤立情况下无效。在推理时间计算成本低廉、自适应计算可用，或目标是研究循环推理而非预训练效率的场景中，更大的 R 可能有用。然而，对于前沿规模的预训练，主要约束是总计算量。在此约束下，$R = 2$ 提供了我们在迭代细化、吞吐量和计算匹配扩展之间观察到的最佳权衡。

3 预训练

我们分两个阶段对 Loopie 模型进行预训练。第一阶段，我们在 3T 个标记上从头开始进行大规模预训练。第二阶段，我们使用 1.26T 个标记进行高质量数据退火，重点放在高质量合成数据、STEM、代码、数学推理和网络数据上。所有 Loopie 模型均使用 Qwen3 模型系列（Qwen Team, 2025）的分词器。我们使用 Megatron-LM（Shoeybi et al., 2019）框架进行所有预训练。

3.1 评估

我们使用 LM Evaluation Harness 框架（Gao et al., 2024）评估预训练检查点。对于第 2 节中的所有实验，我们报告以下八个基准的平均得分：ARC-Challenge（Clark et al., 2018）、ARC-Easy（Clark et al., 2018）、BoolQ（Clark et al., 2019）、CommonsenseQA（Talmor et al., 2019）、HellaSwag（Zellers et al., 2019）、MMLU（Hendrycks et al., 2021）、OpenBookQA（Mihaylov et al., 2018）和 WinoGrande（Sakaguchi et al., 2020）。

3.2 初始化

设 d 表示模型宽度。我们将标记嵌入矩阵 E 和语言建模头 W_{lm} ——或使用权重绑定时共享的嵌入/反嵌入矩阵——初始化为 $E_{ij}, (W_{\text{lm}})_{ij} \sim \mathcal{N}(0, d^{-1})$。在输入侧，查找到的嵌入在进入残差流之前乘以 $\sqrt{d}$，使得嵌入捷径具有 $O(1)$ 激活尺度，而相同的 d^{-1} 方差确保初始 logits $h^{\top} w$ 对于归一化隐藏状态具有 $O(1)$ 方差（Shazeer and Stern, 2018; Takase et al., 2025）。对于 Transformer 块内部的隐藏到隐藏参数，包括注意力投影和前馈层，我们使用 SmallInit：

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{5d}\right), \qquad \operatorname{std}(W) = \frac{1}{\sqrt{2.5d}}.$$

该尺度对应于扩展比为 4 的变换器前馈网络的 Xavier 扇入/扇出初始化，因为 $d_{\text{in}} + d_{\text{out}} = d + 4d = 5d$。我们也将该尺度应用于注意力投影，以降低残差分支子层的尺度（Glorot 和 Bengio，2010；Nguyen 和 Salazar，2019）。因此，我们对嵌入/反嵌入使用 $d^{-1/2}$，它们决定捷径和对数尺度，但对块子层使用 $(2.5d)^{-1/2}$，其雅可比矩阵应保持较小以实现稳定的预训练（Takase 等人，2025）。

3.3 学习计划

我们使用 AdamW（AdamW）优化所有模型（Kingma 和 Ba，2015；Loshchilov 和 Hutter，2019）。我们将 Loopie-6B-A0.6B 的峰值学习率设置为 5×10^{-4}，将 Loopie-20B-A2B 的峰值学习率设置为 3×10^{-4}。我们将 AdamW 超参数设置为 $\beta_1 = 0.9, \beta_2 = 0.95$ 和 $\epsilon = 10^{-15}$，使用权重衰减 0.1，并应用全局梯度范数裁剪，阈值为 1.0。我们采用预热后稳定不变的学习率调度。在指定数量的预热步骤之后——Loopie-20B-A2B 为 6,000 步，Loopie-6B-A0.6B 为 2,000 步——学习率达到峰值，然后保持不变，不再衰减。这种无衰减的稳定阶段在预热后保持较高的有效更新尺度，避免了在退火调度下可能出现的后期高质量数据影响力减弱的问题（Wen 等，2024；Luo 等，2025；Yano 等，2026）。在第二阶段的高质量退火阶段，我们也使用恒定学习率，以最大化从

最高质量退火数据中的学习（Luo 等，2025）。全局批大小为 1024，序列长度为 8192。

3.4 阶段 1：多轮次高质量预训练

在阶段 1 中，我们在 Nemotron-CC-v2-HQ（NVIDIA，2024）上训练 Loopie-20B-A2B 和 Loopie-6B-A0.6B，该数据集是 Nemotron-CC 的高质量子集。语料库包含约 570B 个唯一标记，我们训练四个轮次，总计约 2.28T 个训练标记。这一设计源于我们之前的观察：在高质量数据上重复训练可能比在更大但平均质量较低的语料库上进行单遍训练更有效（Gao 等，2025c）。

3.5 阶段 2：高质量退火

在阶段 2 中，我们从 Nemotron 预训练数据集构建了一个高质量退火混合数据池。最终数据池包含约 1.26T 个标记。该混合数据结合了高质量 SFT 风格数据、合成推理数据、代码数据、合成网络数据和数学数据。数据池总共包含约 1263B 个标记：351B 来自 Nemotron-pre-training-SFT-v1（27.8%），277B 来自 Nemotron-pre-training-Specialized-v1（21.9%），262B 来自 Nemotron-pre-training-Code-v2 的 60% 样本（20.7%），197B 来自 Nemotron-CC-v2-HQ-Synthetic 的 16% 样本（15.6%），126B 来自质量分数为 $\geq 4\ (10.0\%)$, 的 Nemotron-CC-Math-v1，25B 来自 Nemotron-CC-v2.1-HQ（2.0%），以及 25B 来自 Nemotron-CC-v2.1-HQ-Synthetic（2.0%）。图 7 展示了这些相对比例。

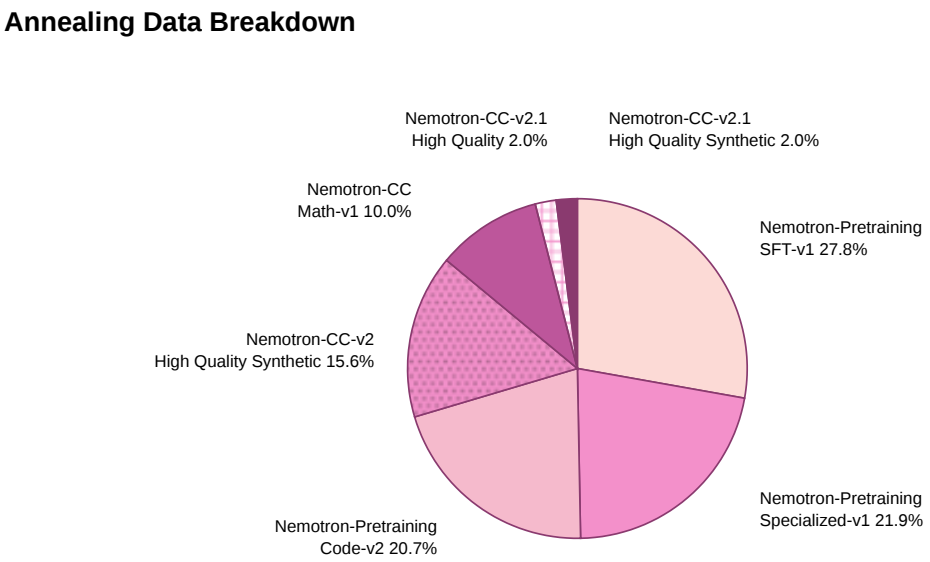


图 7 阶段 2 高质量退火数据池的组成：多个数据源构成了 1.26T 标记的退火配方。

Nemotron-pre-training-SFT-v1。我们纳入了来自 **Nemotron-pre-training-SFT-v1** 的约 3510 亿个标记，这是一个多样化的 SFT 风格数据集，包含合成和精选示例，涵盖 STEM、学术、代码、数学和推理领域，并具有多语言覆盖。其 STEM 部分通过使用 Qwen3 和 DeepSeek 模型进行迭代生成，从高质量的数学和科学种子扩展而来，产生了更困难且更多样化的问题及解答。该数据集还包含从本科和研究生水平文本合成的学术问答对，以及 MMLU 风格的通用问答和基础推理数据。

Nemotron-pre-training-Specialized-v1。我们纳入了来自 **Nemotron-pre-training-Specialized-v1** 的约 2770 亿个标记，该数据集包含用于 STEM 推理、科学编码和跨领域编码的合成数据，以及合成维基百科数据和合成数学教科书数据。STEM 推理部分包括从高级科学种子文档生成的推理性问答演示，而科学编码和跨领域编码子集则引入了研究生或研究级别的编程任务及结构化解决方案。该数据旨在在退火阶段加强科学推理、数学抽象和代码生成。

Nemotron-pre-training-Code-v2。我们随机抽取 **Nemotron-pre-training-Code-v2** 的 60%，得到约 2620 亿个标记。该数据集结合了最新的 GitHub 源代码、合成代码基础问答数据、学生-教师对话、代码审查对话以及由大语言模型重写或转译的源代码。重写和转译组件旨在通过增加风格多样性并让模型接触语义等价的代码变体来改进下游代码生成。

高质量合成网络数据。我们随机抽取 Nemotron-CC-v2-High-Quality-Synthetic 的 16%，得到约 1970 亿个标记。该数据集源自 Nemotron-CC-v2，包含通过 Qwen3-30B-A3B 进行合成改写而增强的英文网络抓取文档。我们还纳入了来自 Nemotron-CC-v2.1-High-Quality-Synthetic 的 250 亿个标记，该数据集通过更新的 Common Crawl 快照和额外改写的中高质量文档扩展了合成高质量网络语料库。

高质量网络数据。我们纳入了来自 Nemotron-CC-v2.1-High-Quality 的约 250 亿个 token。该子集包含近期的 Common Crawl 快照以及从多种语言翻译成英语的高质量数据。额外的基于大语言模型的过滤移除了信息量不足的翻译文档。纳入这些数据有助于在退火阶段保持对自然网络文本的接触。

数学数据。最后，我们纳入了来自 Nemotron-CC-Math-v1 的约 1260 亿个 token，仅使用质量评分为 4 分及以上的文档。Nemotron-CC-Math-v1 是一个高质量的数学预训练语料库，基于 Common Crawl 构建，其处理流程旨在保留方程和代码、将数学符号转换为标准 LaTeX 格式并去除噪声。纳入该部分是为了增强数学推理和符号问题求解能力。

4 后训练

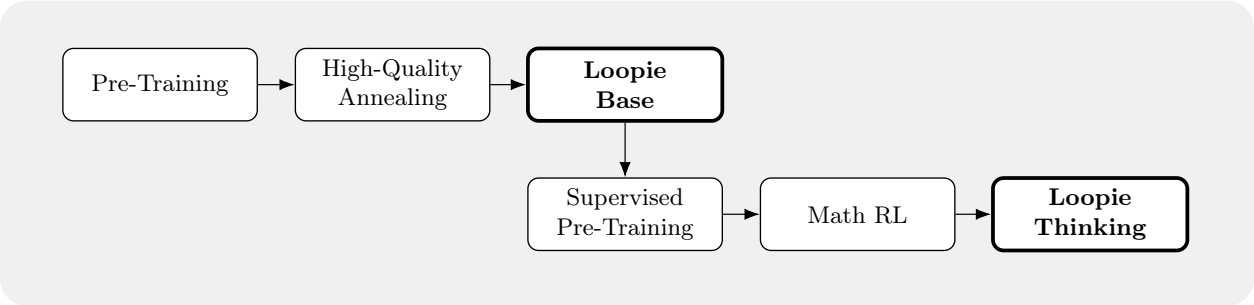


图 8 Loopie 训练流程概览。模型首先经过预训练和退火以生成 **Loopie** Base，随后进行监督式预训练和数学强化学习以生成 **Loopie** Thinking。

Loopie 的后训练方案在第二阶段高质量退火之后包含两个主要阶段。首先，我们引入监督式预训练阶段，在该阶段中，我们继续在 2 万亿个 token 的指令遵循、推理、编码、数学和工具使用数据上训练模型，使基础模型获得广泛的任务遵循和问题求解能力，同时保留预训练期间学到的知识和通用能力。其次，我们应用大规模强化学习来进一步增强 Loopie 的

推理能力，改进其长程问题求解能力，并对模型进行对齐，使其产生可靠的思维迹。这两个阶段共同将 Loopie Base 转化为最终的 Loopie Thinking 模型。

4.1 监督式预训练

我们引入了监督式预训练（Supervised Pre-training，SPT），这是一种训练机制，它将监督式微调（Supervised Fine-Tuning，SFT）的监督模式与语言模型预训练（Pre-training，PT）的优化规模相结合。SPT 使用与 SFT 完全相同的训练数据和词元级目标：提示词和上下文词元不参与损失计算，只有监督目标词元对优化有贡献。然而，与传统的 SFT 不同，SPT 采用语言模型预训练中典型的全局批大小、序列长度和词元预算。

表 2 监督式预训练（SPT）、传统监督式微调（SFT）和预训练（PT）的比较。SPT 列报告了用于 Loopie 的 $\uparrow, \downarrow$ 的代表性配置，$\approx$ 分别表示相对于相应起始检查点的改进、退化和无实质性变化。稳定性条目总结了我们在评估训练周期内的观察结果，而非普遍保证。

	SPT	SFT	PT
Loss function	Cross-entropy	Cross-entropy	Cross-entropy
Loss-bearing positions	Supervised target tokens only	Supervised target tokens only	All non-padding tokens
Pre-training metrics	↑	↓	↑
Reasoning metrics	↑	↑	≈
Overfitting after multiple epochs	Not observed	Observed	Not observed
Global batch size	≥ 1,024	32–128	≥ 1,024
Sequence length	≥ 128 K	8–32 K	4–8 K
Nominal token positions per batch	≥ 128 M	128 K–1 M	8–32 M
Total training token budget	≥ 2 T	10–100 B	≥ 4 T

SPT 将两个通常耦合的设计选择分离开来：（i）哪些词元对训练损失有贡献，以及（ii）执行优化的规模。SFT 和 SPT 共享前者，而 PT 和 SPT 共享后者。因此，SPT 并非在 SFT 损失和 PT 损失之间进行插值。相反，它在 PT 规模的优化机制中应用 SFT 风格的监督目标。

训练目标。考虑一个监督示例 $z_i = [c_i; y_i]$，其中 c_i 表示输入上下文，y_i 表示监督目标响应。设 $w_{i,t} \in \{0, 1\}$ 为位置 t 处词元的二元损失掩码。所有三种训练机制都使用词元级交叉熵：

$$\mathcal{L}_{\mathrm{CE}}(\theta; w) = -\frac{\sum_{i=1}^{B}\sum_{t=1}^{T_i} w_{i,t} \log p_\theta\left(z_{i,t} \mid z_{i,<t}\right)}{\sum_{i=1}^{B}\sum_{t=1}^{T_i} w_{i,t}}.$$

对于 SPT 和 SFT，仅当 $z_{i,t}$ 属于监督目标 y_i 时，$w_{i,t} = 1$ 才为真；提示词、上下文和填充位置被屏蔽。相比之下，在传统 PT 中，$w_{i,t} = 1$ 适用于每个非填充词元。因此，所有三种机制都采用相同的交叉熵损失，但在损失应用的位置和优化规模上有所不同。

表 2 总结了这些区别。对于代表性的 SFT 设置，包括全局批大小、训练词元数和序列长度，我们主要遵循 OLMo 3、Nemotron 3 系列和 Step-3.5-Flash 所报告配置（Team OLMo, 2026; NVIDIA, 2025; Bercovich et al., 2025; NVIDIA, 2026a,b; Huang et al., 2026a）。我们使用预训练度量来指代评估通用基础模型的度量

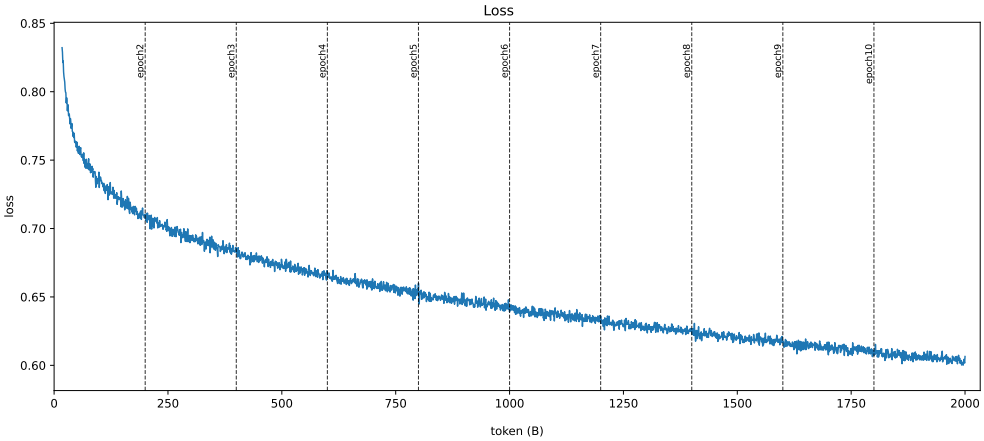


图 9 Loopie-6B-A0.6B 在监督式预训练期间的训练损失。由于评估损失曲线与训练损失曲线几乎重合，因此仅展示训练损失曲线。在总计 2 万亿个词元上进行 10 个训练轮次的监督式预训练过程中，损失平稳下降，且在训练轮次边界处未观察到损失悬崖现象。

能力与推理度量指主要评估挑战性推理任务性能的度量，例如竞赛级数学与编程问题。在我们的实验中，监督式预训练同时提升两类度量的性能。相比之下，传统的监督式微调基线仅提升推理度量的性能，却降低预训练度量的性能；而预训练提升预训练度量的性能，同时使推理度量的性能大致保持不变。

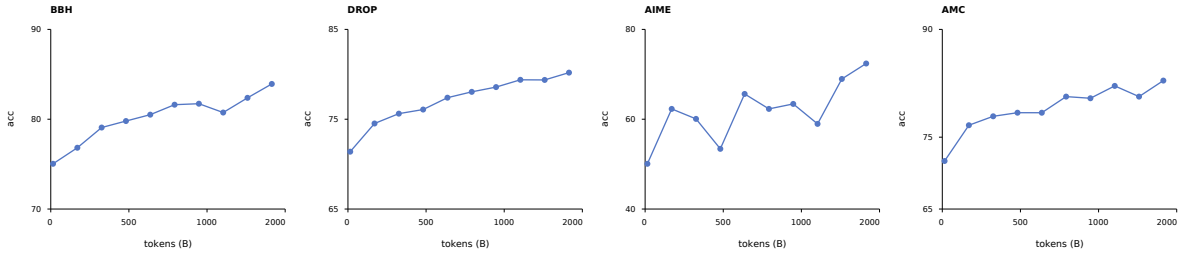


图 10 监督式预训练期间推理度量的变化趋势。在 2 万亿个词元的训练过程中，推理度量的性能持续提升，且未出现放缓迹象。

对监督数据进行预训练规模的优化。监督式预训练每个全局批次处理 1.28 亿个词元，约为传统监督式微调每批次处理量的 1000 倍。这使得监督式预训练相比标准监督式微调更不易发生过拟合。该规模赋予监督式预训练独特的训练动态，这与在大语言模型训练机制中优化超参数与扩展行为可能发生变化的观察结果一致（Jin 等人，2023）。在监督式预训练下，模型可以平稳地训练更多轮次，而不会出现过拟合迹象，例如训练轮次边界处损失突然下降。同时，下游度量的性能持续稳步提升。此外，监督式预训练缓解了灾难性遗忘（Wu 等人，2024），这是传统监督式微调的主要缺陷，通常会导致预训练期间获得的通用知识大幅退化。值得注意的是，监督式预训练不仅避免了这种退化，还进一步提升了通用知识性能。如图 11 所示，在 2 万亿个词元上约 10 个训练轮次的训练过程中，监督式预训练持续提升下游预训练度量的性能，包括 ARC-Challenge 与 MMLU。这一发现挑战了监督式微调会导致灾难性遗忘的传统观点。

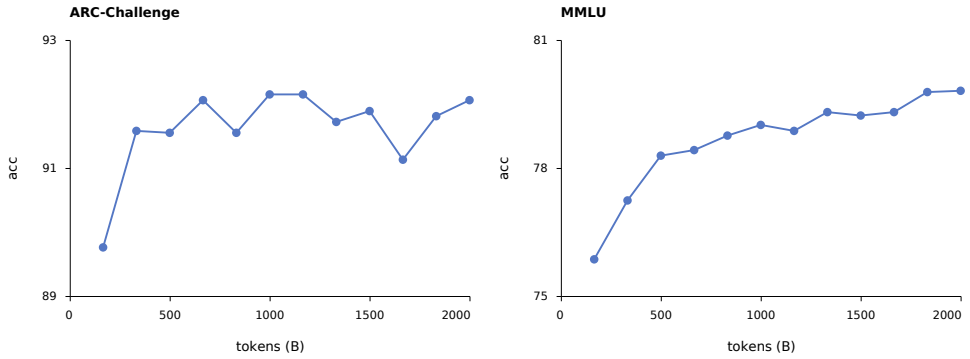


图 11 监督预训练期间预训练指标的趋势。ARC-Challenge 代表通用推理性能，而 MMLU 代表通用知识性能。在 2 万亿个标记的训练过程中，预训练指标的性能并未下降，反而持续提升。

简单地以更多训练轮次运行传统 SFT 配置并不能复现 SPT 的优化机制。传统规模的 SFT 通常在第二至第四个训练轮次之间开始过拟合。在小批量 SFT 中，每个训练轮次涉及从小批量计算出的多次参数更新，重复接触相同样本可能导致快速特化和记忆化。相比之下，每次 SPT 更新聚合了来自一千多个序列和超过一亿个名义标记位置的监督信号。我们假设，这种广泛的梯度聚合，加上长上下文和大量标记预算，缓解了重复 SFT 中常见的过度狭窄特化现象。

4.2 强化学习

在 SPT 之后，我们应用强化学习阶段以获得 Loopie Thinking。遵循先前工作，我们在训练期间不混合来自多个领域的数据，因为领域特定的长度偏差可能相互干扰（Chen 等，2026c；Liu 等，2026）。我们首先在数学任务上进行强化学习；一旦性能饱和，我们继续在编码任务上进行强化学习。

算法。我们的优化器基于分组序列策略优化（GSPO）（Zheng 等，2025），并采用了 DAPO（Yu 等，2025）引入的非对称裁剪和动态采样技术。对于每个提示 q，我们从旧策略 $\pi_{\theta_{\text{old}}}$ 中采样一组 G 个补全 $\{o_i\}_{i=1}^{G}$，并使用验证器对每个补全进行评分，得到结果奖励 $\{R_i\}_{i=1}^{G}$。我们通过对采样组内的奖励进行归一化来估计序列级优势：

$$\hat{A}_i = \frac{R_i - \text{mean}(\{R_j\}_{j=1}^{G})}{\text{std}(\{R_j\}_{j=1}^{G}) + \epsilon},$$

其中 ϵ 是一个小的数值常数。与标记级策略优化不同，GSPO 定义了长度归一化的序列级重要性比率：

$$s_i(\theta) = \left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\theta_{\text{old}}}(o_i \mid q)} \right)^{\frac{1}{|o_i|}} = \exp\left(\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \frac{\pi_\theta(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})} \right).$$

也就是说，$s_i(\theta)$ 是响应上标记级重要性比率的几何平均值。长度归一化降低了序列级比率的方差，并使不同长度的响应保持在可比较的数值范围内。策略使用序列级裁剪目标进行优化：

$$\mathcal{J}_{\text{RL}}^{\text{GSPO}}(\theta) = \mathbb{E}_{q\sim\mathcal{D},\,\{o_i\}_{i=1}^{G}\sim\pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{G} \sum_{i=1}^{G} \min\left(s_i(\theta)\hat{A}_i,\ \text{clip}\left(s_i(\theta), 1-\epsilon_{\text{low}}, 1+\epsilon_{\text{high}}\right)\hat{A}_i \right) \right].$$

与词元级裁剪不同，GSPO 将裁剪操作应用于整个响应。因此，同一响应中的所有词元共享相同的序列级重要性权重，使离策略校正和优化的单位与奖励的单位保持一致。遵循裁剪上限策略，我们使用 $\epsilon_{\text{high}} > \epsilon_{\text{low}}$。在 GSPO 下，这种非对称裁剪应用于序列级重要性比率，而非单个词元级比率。较大的上裁剪范围允许具有正优势的探索性响应获得更强的概率增加更新，而更保守的下裁剪范围则限制了具有负优势的响应的过度概率降低。我们进一步应用提示级动态过滤。仅当采样的组中同时包含成功和失败的完成结果时，才保留该提示：

$$0 < \sum_{i=1}^{G} \mathbf{1}\{R_i = R_{\text{pass}}\} < G.$$

完全正确或完全错误的组无法为 GRPO 提供有用的相对偏好信号。因此，我们对候选提示进行过采样，并过滤掉此类组，直到有效批次填满。这使优化集中于产生非退化策略梯度的提示。

训练数据。我们使用 Guru-RL 语料库（Cheng 等人，2025）中的数学和代码子集。数学池来源于 OR1、DAPO 和 DeepScaleR 来源（He 等人，2025；Yu 等人，2025；Tan 等人，2025）。代码池结合了 LeetCodeDataset、TACO-Verified、PrimeIntellect/SYNTHETIC-1 以及历史 LiveCodeBench 训练问题（Xia 等人，2025；Li 等人，2023a；Li，2025；Mattern 等人，2025；Jain 等人，2024）。我们应用了额外的筛选流程，移除格式错误的提示、无法验证的答案、不稳定的测试、重复示例以及与保留评估集高度重叠的示例。数学问题的奖励使用基于规则的答案等价性计算，而编码问题的奖励使用沙箱化单元测试执行计算。

训练计划。我们分两个上下文长度阶段进行训练。第一阶段使用最大响应长度为 32K 个标记。该阶段计算效率高，因为大多数展开在此限制内自然终止。当展开截断率超过 10% 时，我们切换到 64K 标记阶段，为策略提供额外空间以进行更长的推导和代码推理轨迹。我们持续进行强化学习，直到聚合验证分数停止提升并开始下降；在持续退化之前紧邻的检查点被选为 Loopie Thinking。在整个训练过程中，我们监控验证准确率、平均响应长度、生成熵和截断率，以检测后期过度优化。

4.3 结果

我们报告最终 Loopie Thinking 模型的两个互补比较。我们将 Loopie-20B-A2B 与具有相似激活参数规模的混合专家推理模型进行比较，并将 Loopie-6B-A0.6B 与更广泛的紧凑推理模型集进行比较。本节中的所有评估均使用 EvalScope 框架进行，IFEval 分数在 inst_level_loose 设置下报告。对于 AIME 2024 和 AIME 2025，我们报告 avg@8 结果；对于所有其他基准，我们报告 pass@1。预告图中显示的 AIME 结果是针对 AIME 2024 的。

表 3 Loopie-20B-A2B 与相似规模的混合专家推理模型在知识、通用、代码和数学基准上的比较。

	Qwen3 30B-A3B Thinking	Nemotron 3 Nano 30B-A3B	Nemotron Cascade 2 30B-A3B	GPT-OSS 20B-A2B High	Loopie 20B-A2B Thinking
Pre-training tokens	36T	25T	25T	Unknown	**3.5T**
Knowledge					
MMLU	85.83	80.52	81.22	81.64	81.28
MMLU-Redux	88.25	82.54	83.89	83.40	83.61
General					
ARC-Challenge	96.67	91.96	93.86	92.42	93.52
DROP	87.70	85.92	79.06	62.82	82.08
BBH	86.59	68.76	75.86	84.76	82.28
SciQ	95.40	93.40	92.40	93.80	92.50
IFEval	85.58	77.05	79.21	70.03	84.72
Code					
MBPP	96.50	90.66	80.54	78.99	89.49
MBPP+	98.68	91.53	83.60	97.62	83.07
HumanEval	96.95	86.59	79.88	92.07	89.02
HumanEval+	92.07	87.80	78.66	89.02	87.20
Math					
AIME 24	90.10	85.00	93.33	88.33	92.09
AIME 25	83.75	74.17	91.25	87.50	83.75
AMC	96.73	91.80	93.57	91.05	94.21
OlympiadBench	81.20	76.68	88.64	70.03	80.50

表 3 突出了 Loopie-20B-A2B 的预训练效率。Loopie 仅在 3.5 万亿个标记上进行预训练，而 Nemotron 3 Nano 和 Nemotron Cascade 2 各自在来自同一预训练数据源的 25 万亿个标记上进行训练。因此，Loopie 使用的预训练标记不到它们的七分之一，却在大多数知识和通用能力基准上达到或超越了这两个模型。特别是，Loopie 在 MMLU 上达到 81.28 分，超过了 Nemotron 3 Nano（80.52 分）和 Nemotron Cascade 2（81.22 分）。在 ARC-Challenge 上，Loopie 得分为 93.52 分，比 Nemotron 3 Nano 高出 1.56 分，与 Nemotron Cascade 2 仅差 0.34 分。在 BBH 上，Loopie 也大幅超越这两个模型，得分为 82.28 分，而后者分别为 68.76 分和 75.86 分；在 IFEval 上，Loopie 得分为 84.72 分，而后者分别为 77.05 分和 79.21 分。这一优势延伸到更广泛的预训练评估套件中：在所有报告的知识或通用能力基准上，Loopie 至少超过了两个 Nemotron 模型中的一个，并在七个基准中的五个上分别超越了它们，尽管其预训练标记预算小了七倍以上。因此，由此产生的模型概况不仅仅是在更小的训练预算下具有竞争力；它表明在相同的预训练数据下，标记效率显著更高。Loopie 还保持了强大的后训练推理性能，在 AIME 2024 上达到 92.09 分，在 AMC 上达到 94.21 分，在这两个基准上均位列所列模型的第二名，而其 AIME 2025 得分为 83.75 分，与 Qwen3-30B-A3B Thinking 持平。

	DeepSeek-R1 Distill-Qwen 1.5B	Gemma-4 E2B-it	Gemma-4 E4B-it	Qwen3 1.7B	MiniCPM5 1B	Ouro 1.4B Thinking	Ouro 2.6B Thinking	Loopie 6B-A0.6B
知识								
MMLU	44.85	63.33	72.97	69.08	61.54	72.40	82.70	78.36
MMLU-Redux	51.28	72.28	79.61	74.28	70.63	73.75	86.28	81.35
General								
ARC-Easy	69.19	87.08	92.97	91.20	85.27	90.87	94.28	93.77
ARC-Challenge	61.09	83.28	90.02	86.09	75.34	88.48	93.94	91.13
SciQ	44.10	86.80	94.40	87.70	82.48	91.60	94.90	90.10
Code								
MBPP	35.80	77.04	82.49	80.54	50.97	88.63	95.69	76.65
MBPP+	63.76	86.77	90.21	82.80	81.75	89.89	96.01	84.66
HumanEval	65.85	79.27	91.46	85.98	86.59	95.12	95.12	84.15
HumanEval+	65.24	81.71	85.98	82.93	86.59	87.80	89.02	79.88
Math								
AIME 24	35.83	38.33	49.17	49.58	47.50	50.83	62.50	80.42
AIME 25	24.17	26.25	35.42	35.00	38.75	44.17	51.67	70.83
AMC	67.91	75.37	85.08	73.88	28.36	78.36	85.82	84.33
MATH-500	83.60	89.00	91.60	90.60	60.00	92.20	92.20	93.80
GSM8K	72.18	90.60	93.40	90.14	76.35	94.09	96.13	93.63

5 相关工作

5.1 循环变换器

本文使用术语“循环模型”来指代一种架构，该架构在单次前向计算中重复使用学习到的内部算子，而非反复调用一个原本完整的模型。这一设计谱系早于当代语言模型：神经 GPU 反复应用共享的卷积转移来学习算法，自适应计算时间（ACT）学习执行多少次循环更新，通用变换器（UTs）在深度上绑定自注意力和转移模块，而深度均衡模型（DEQs）通过求解权重绑定变换的不动点来避免显式展开（Kaiser and Sutskever, 2016; Graves, 2016; Dehghani et al., 2019; Bai et al., 2019, 2020）。关于训练隐式模型和通用神经算法学习器的工作进一步建立了实用的优化方法和可复用的处理器，用于有效无界或任务共享的计算。这些是循环变换器的直接架构前身，而不仅仅是通用的循环或测试时计算方法。在变换器家族中，层绑定、循环状态、自适应深度和记忆复用也出现在 ALBERT、绑定变换器、深度自适应变换器、反馈记忆、循环记忆变换器和块循环变换器中（Lan et al., 2020; Xia et al., 2019; Elbayad et al., 2020; Fan et al., 2020; Bulatov et al., 2022; Hutchins et al., 2022）。现代变体将共享深度计算本身作为核心扩展轴。MoEUT 循环地复用细粒度专家组，松弛递归变换器用深度特定的低秩适配器补充重复块，隐式状态空间语言模型迭代共享转移趋向不动点，同时保持显著的训练并行性（Csordás et al., 2024）。因此，循环变换器不依赖普通的深度扩展，而是学习一个可复用的转移 $h_{t+1} = F_\theta(h_t, x, e_t)$，其中 e_t 可以编码循环索引、停止状态或注入的输入副本。有效的推理深度随后可以变化，而无需在每一步引入独立参数化的层。

5.2 归纳偏置

循环深度施加了若干偏差，这些偏差在独立变换器（Transformer）层的堆叠中较弱或不存在。首先，循环算子共享鼓励可复用的更新规则，而非一系列层特定的特征映射。这是通用变换器（UT）及绑定深度模型所特有的递归偏差，理论分析将其与不动点迭代、基于梯度的上下文学习、归一化梯度下降及幂迭代联系起来（Dehghani 等，2019；Takase 和 Kiyono，2023；Yang 等，2024；Gatmiry 等，2024a；Chen 等，2025；Wu 等，2026）。其次，共享深度将有效计算与参数数量解耦：一个紧凑的循环核心可以被展开更多步，从而更倾向于迭代细化，而非在每一层中记忆单独的计算（Lan 等，2020；Saunshi 等，2025；Geiping 等，2025；Zhu 等，2025；Schwethelm 等，2026）。

第三，循环会引入迭代算法偏差。当目标计算自然由小规则集的重复应用表达时，循环模型可以模拟程序、优化过程、图算法和结构化算法处理器，通常在循环次数适应问题规模时改善长度或深度外推（Giannou 等，2023；Back de Luca 和 Fountoulakis，2024；Gao 等，2025a；Fan 等，2025；Xu 和 Sato，2025a）。第四，权重共享创造了知识重新访问和组合性偏差：在推断出每个新的桥接实体后，可以再次应用相同的检索、绑定和更新机制。这直接针对普通变换器中隐式多跳组合的失败，并激励了携带离散和连续状态或对齐潜在和显式推理的架构（Wang 等，2024；Biran 等，2024；Guo 等，2025；Yao 等，2025；Kohli 等，2026；Fu 等，2026a；Fan 等，2026）。第五，循环提供了潜在草稿和自适应计算偏差。额外计算发生在隐藏状态中，而不是通过生成额外的语言标记，而时间步条件化、逐标记路由、动态停止、不动点收敛和捷径一致性为更难的输入分配更多计算（Saunshi 等，2025；Geiping 等，2025；Bae 等，2025；Fu 等，2025；Jeddi 等，2026；Movahedi 等，2026）。这些好处并非自动获得：循环模型可能会漂移、过度思考或退化为浅层计算，除非其状态注入、归一化、监督和停止规则被设计用于稳定迭代（Chowdhury 和 Caragea，2024；Labovich，2026；Rauba 等，2026；Moosa 等，2026）。

5.3 理论、机制与扩展

形式化结果刻画了共享深度计算的能力与极限。循环变换器能够执行指令级程序、模拟潜在思维链，并通过时间步调制获得更优的逼近率（Giannou 等，2023；Saunshi 等，2025；Xu 和 Sato，2025b）。更专门的分析表明，循环复用的注意力层可以学习上下文逻辑回归的归一化梯度更新，或在层归一化下实现幂法（Wu 等，2026）。同时，压缩的循环状态相对于全序列状态思维链方法产生了记忆预算分离，而集合值或图计算需要仔细区分不动点、收敛与停机。互补的强化学习分析形式化了额外内部计算如何改变计算受限策略的类别。

第二类工作研究循环次数下的优化、稳定性与扩展性。不动点分析确定了循环状态保持可达、输入依赖和可训练的条件；残差缩放理论认为相关的绑定块更新需要循环感知的缩放；等深度研究量化了循环遍数、唯一深度与训练计算之间的交换率（Labovich，2026；Wang 等，2026b；Schwethelm 等，2026）。双尺度潜在动力学分析推导出对收敛敏感的早停准则，而自适应计算的受控研究表明，学习到的循环分配可以与词元难度相关，但不一定能外推到更长的输入（Pappone 等，2025；Moosa 等，2026）。记忆词元和 ACT 初始化可以决定通用变换器是否进入非平凡的推理状态，而分层循环与扁平循环即使在共享权重控制下也可能表现出显著不同的优化行为（Sapunov，2026；Han，2026）。压缩研究同样表明，如果量化或剪枝破坏了精确解所需的递归轨迹，那么保持每单元准确率或局部状态是不够的。

机制性工作考察循环步骤中表示的内容。探测研究发现，循环语言模型通常接近循环特定的不动点或重放类似前馈的推理阶段，而

关于字面潜在思维链的证据是混合的且依赖于深度（Blayney 等人，2026；Lu 等人，2025）。共享的循环模块仍然可以通过不对称的状态身份专门化以执行不同的功能角色，并且交互局部性测量揭示了重复的局部更新如何累积成全局谜题结构。其他探针从循环状态之间的差异中恢复关系偏好信息，而对表格基础模型的分析利用观察到的层冗余来激励单个重复应用的层。最后，循环深度模型可以解锁系统的多跳泛化，但自回归控制表明，其益处取决于计算放置的位置，而不仅仅是递归本身（Kohli 等人，2026；Rauba 等人，2026）。

5.4 架构、训练目标与推理

近期循环语言模型将递归视为参数数量与词元数量之外的第三条缩放轴。Huginn 与 Ouro 预训练了递归深度语言模型，其共享核心可在测试时展开为可变深度，而改造式递归与 LoopUS 则将预训练的前馈模型转换为潜在细化系统，无需完全从头训练递归模型（Geiping 等，2025；Zhu 等，2025；McLeish 等，2025；Park 等，2026）。HRM-Text 与 CHERRY 探索了深度语言模型的递归压缩，Hyperloop 仅重复具有跨循环连接的中间块，而 LoopMoE、稀疏循环层与通用专家池则将迭代深度与条件容量相结合（Wang 等，2026a；Kwon 与 Park，2026；Zeitoun 等，2026；Chen 等，2026b；Lee 等，2026）。MoEUT 与松弛递归变换器提供了早期的专家路由与低秩机制，用于在参数共享下恢复表达能力（Csordás 等，2024）。动态深度架构不仅决定如何更新状态，还决定在何处递归以及递归多久。递归混合体将词元路由至不同深度；CoTFormer 暴露较早的递归表示并学习计算预算路由器；AdaPonderLM 与 Think-at-Hard 选择性地分配额外潜在迭代；而层链方法则在测试时跳过或重复预训练层（Bae 等，2025；Mohtashami 等，2025；Fu 等，2025；Li 等，2026）。LoopFormer 对采样深度上的轨迹进行正则化，自适应循环模型将停止机制与外部记忆耦合，而 ChainGPT、MoDr 与深度递归注意力混合体通过多秩更新、分支路由或序列与深度注意力的混合来丰富递归状态转移（Jeddi 等，2026；Frey 等，2026a；Zheng 等，2026；Zhang 等，2026）。SpiralFormer 则改变重复应用中的分辨率调度，而子目标持久性模型研究分层推理器应在何时重新规划，而非在每一步更新相同的潜在计划（Yu 等，2026；Chadha，2026）。另一条并行工作线针对稳定且高效的展开。并行循环变换器在循环间共享表示，而 LT2 以线性时间循环设计取代二次注意力（Wu 等，2025；Deng 等，2026）。MELT 使缓存内存独立于循环深度；LASER 在训练期间压缩递归激活；而 Hyperloop 与 CHERRY 减少了不同参数块的数量（Çakar 等，2026；Zeitoun 等，2026；Kwon 与 Park，2026）。Parcae 约束递归动态并推导缩放定律，CART 将每次迭代锚定于上下文，全循环信号路由稳定高循环计数，随机停止改善对未见深度的外推，而稳定性感知的递归训练减少测试时缩放下的漂移（Prairie 等，2026；Capps，2026；Fu 等，2026b；Kuo 等，2026）。免训练循环探索预训练模型能否在不进行架构重训练的情况下被递归重用（Chen 等，2026a）。若干方法监督轨迹而非仅监督最终答案。LoopRPT 在潜在迭代上应用强化预训练，RLTT 在潜在轨迹上分配奖励，去噪递归训练模型反复纠正损坏目标，而生成式递归推理将递归扩展为随机多轨迹生成（Tang 等，2026；Cameron 等，2026；Baek 等，2026）。概率 TRM 在推理时注入噪声并在递归轨迹间进行选择，而 LoopFormer 使用捷径一致性来对齐不同计算预算（Sghaier 等，2026；Jeddi 等，2026）。循环扩散语言模型、定点掩码生成模型、均衡推理器与吸引子模型将显式递归与去噪或基于收敛的生成联系起来（Miele 等，2026；Fein-Ashley 与 Rashidinejad，2026）。这些方法使中间状态质量、收敛几何与轨迹多样性成为首要训练目标，而非深度的附带副产品。

5.5 上下文、算法与组合泛化

上下文学习可被解释为在演示上执行隐式学习算法。贝叶斯、梯度下降、预条件梯度、线性模型及因果结构等解释促使研究者探究每个循环步骤是否实现一次额外的内部更新（Xie 等，2022；Akyurek 等，2023；von Oswald 等，2023；Ahn 等，2023；Li 等，2023b；Zhang 等，2023；Nichani 等，2024）。循环变换器使这种对应关系显式化：它们以显著更少的独特参数学习数据拟合算法，实现多步梯度过程，并可使用不同的预处理、循环和后处理阶段（Yang 等，2024；Gatmiry 等，2024a；Chen 等，2025；Gao 等，2025a）。程序仿真结果和归一化梯度分析进一步将共享深度与可复用的算法原语联系起来（Giannou 等，2023）。同样的先验与长度和组合外推相关。神经 GPU 和通用循环处理器学习重复的算法更新，而循环变换器仿真图算法，并在循环步数随实例规模增长时获得强大的长度泛化能力（Kaiser 和 Sutskever，2016；Back de Luca 和 Fountoulakis，2024；Fan 等，2025）。时间步调制、多分辨率递归和深度循环组合模型解决了单一静态更新在远超训练深度后失效的问题（Xu 和 Sato，2025b；Yu 等，2026；Chen，2026）。DiscoLoop 在跳步间传递离散嵌入和连续隐藏状态，LOTUS 将循环潜在块与显式思维链计算对齐（Fu 等，2026a；Fan 等，2026）。这些结果补充了普通变换器往往记忆局部事实或短计算，却无法在分布外系统组合它们的证据（Biran 等，2024；Yao 等，2025；Dziri 等，2023）。

5.6 抽象推理

ARC 风格任务是一种自然的压力测试，因为它们要求从少量演示中归纳出新的变换，而非回忆固定技能（Chollet，2019；Chollet 等，2025）。HRM 通过耦合的高层与低层循环模块获得可观的等效深度，TRM 将设计简化为一个微小的重复应用网络，而 URM 研究将大部分增益归因于通用变换器式循环与非线性深度计算（Wang 等，2025；Jolicoeur-Martineau，2025；Gao 等，2025b）。受控分析提醒，层级结构、数据增强、身份条件化、多数投票以及竞赛时自适应都可能实质性地影响报告分数（Ge 等，2025；Ren 和 Liu，2026；Roye-Azar 等，2025；McGovern，2025）。

后续设计探索了改进递归抽象推理的互补途径。CosmicFish-HRM 将层级循环适配到紧凑语言模型；递归推理机推广了生成器-求解器递归；不动点推理器在收敛时停止，而非在预设深度停止（Lakkapragada，2026；Komisarczyk 等，2026；Movahedi 等，2026）。生成式、概率式和去噪递归模型引入了轨迹多样性或迭代破坏-修复课程（Baek 等，2026；Sghaier 等，2026；Cameron 等，2026）。均衡与吸引子推理器学习收敛的解动态，而微型自回归递归模型检验在递归与普通自回归深度之间，相当的计算量是否更值得投入（Fein-Ashley 和 Rashidinejad，2026；Rauba 等，2026）。记忆增强的 UT、LoopViT 以及交互局部性分析揭示了互补的深度-状态、视觉递归和机制层面的视角（Sapunov，2026；Shu 等，2026）。总体而言，这些文献支持将递归作为迭代抽象的架构先验，同时表明成功的测试时扩展依赖于稳定的状态转移和信息丰富的中间监督，而非仅仅循环次数（Chollet 等，2026；Vahdati 等，2026）。

6 未来工作

由于计算资源的限制，我们的后训练实验主要聚焦于数学和代码推理任务。我们并未广泛探索先前工作中研究的其他重要能力，例如科学问答、指令遵循、与人类对话偏好的对齐，或智能体任务求解。未来工作的一个自然方向是继续在更广泛的任务和偏好信号上对 Loopie 进行后训练，这可能进一步提升其实用性和通用能力。此外，由于计算资源有限，我们未能对监督式预训练进行足够全面的消融

研究，我们将其留待未来工作。另外，我们的研究主要关注在预训练期间匹配计算预算，尚未对推理时计算进行系统性研究。匹配和优化推理时计算仍是未来工作的重要方向；先前的工作，如并行循环变换器（Parallel Loop Transformer，Wu 等，2025），为这一方法提供了一个有前景的示例。最后，我们的研究有意将循环变换器应用于相对干净且控制良好的基础架构 Qwen3-30B-A3B，以避免架构变化引入的混杂因素。因此，我们未研究若干可能与本方法互补的最新架构进展。探索 Loopie 与这些新设计的交互仍是未来工作的重要方向。

7 结论

我们提出了 Loopie，一个循环混合专家语言模型家族，在匹配的预训练计算预算下使循环深度具有竞争力。通过将层循环递归与硬件感知的扩展配方相结合，Loopie 在不同模型规模上一致地优于计算匹配的普通变换器基线。基于监督式预训练和强化学习的大规模后训练流程进一步赋予 Loopie 强大的数学推理和编码能力。这些结果表明，当递归计算与架构和训练效率联合优化时，可以作为大语言模型的实用扩展轴。

作者贡献

高子天完成了模型训练、实验实现以及本文的撰写。模型架构由高子天、陈一龙、肖一豪和杨新宇设计，并得到了陶然、周乔伊和戴布赖恩的指导。陈一龙和杨新宇在智求研究所实习期间完成了这项工作。

8 致谢

- 感谢卡内基梅隆大学的黄本浩和清华大学的王少文提出的宝贵反馈和细致审阅。
- 感谢智求研究所的叶正茂提供的基础设施支持。
- 感谢英伟达涅墨特隆对开源社区的杰出贡献。特别是，如果没有涅墨特隆-CC-v2 和涅墨特隆-卡斯卡德-2 提供的训练数据，卢皮斯将无法达到目前的性能水平。
- 感谢艾伦人工智能研究所（AI2）提供的本文所使用的优雅的 LaTeX 模板。

参考文献

S. Abnar, H. Shah, D. Busbridge, A. M. E. Ali, J. Susskind, and V. Thilak. Parameters vs FLOPs: Scaling laws for optimal sparsity for mixture-of-experts language models. In *International Conference on Machine Learning*, 2025. URL https://arxiv.org/abs/2501.12370.

K. Ahn, X. Cheng, H. Daneshmand, and S. Sra. Transformers learn to implement preconditioned gradient descent for in-context learning. In *Advances in Neural Information Processing Systems*, 2023.

E. Akyurek, D. Schuurmans, J. Andreas, T. Ma, and D. Zhou. What learning algorithm is in-context learning? investigations with linear models. In *International Conference on Learning Representations*, 2023.

A. Back de Luca and K. Fountoulakis. Simulation of graph algorithms with looped transformers. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL https://arxiv.org/abs/2402.01107.

S. Bae, Y. Kim, R. Bayat, S. Kim, J. Ha, T. Schuster, A. Fisch, H. Harutyunyan, Z. Ji, A. Courville, and S.-Y. Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *arXiv preprint arXiv:2507.10524*, 2025. URL https://arxiv.org/abs/2507.10524.

J. Baek, M. Jo, M. Kim, M. Ren, Y. Bengio, and S. Ahn. Generative recursive reasoning, 2026. URL https://arxiv.org/abs/2605.19376.

S. Bai, J. Z. Kolter, and V. Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2019.

S. Bai, V. Koltun, and J. Z. Kolter. Multiscale deep equilibrium models. In *Advances in Neural Information Processing Systems*, 2020.

A. Bercovich, I. Levy, I. Golan, et al. Llama-Nemotron: Efficient reasoning models, 2025. URL https://arxiv.org/abs/2505.00949.

E. Biran, D. Gottesman, S. Yang, M. Geva, and A. Globerson. Hopping too late: Exploring the limitations of large language models on multi-hop queries. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14113–14130, Miami, Florida, USA, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-main.781. URL https://aclanthology.org/2024.emnlp-main.781/.

H. Blayney, Álvaro Arroyo, J. Obando-Ceron, P. S. Castro, A. Courville, M. M. Bronstein, and X. Dong. A mechanistic analysis of looped reasoning language models, 2026. URL https://arxiv.org/abs/2604.11791.

A. Bulatov, Y. Kuratov, and M. Burtsev. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, 2022.

C. Cameron, W. Wang, N. Ivanov, A. Bhattacharyya, D. Chételat, and Y. Zhang. One step forward and K steps back: Better reasoning with denoising recursion models, 2026. URL https://arxiv.org/abs/2604.18839.

C. A. Capps. Cart: Context-anchored recurrent transformer – a parameter-efficient architecture with learned stability, 2026. URL https://arxiv.org/abs/2606.01495.

A. Chadha. When to re-plan: Subgoal persistence in hierarchical latent reasoning, 2026. URL https://arxiv.org/abs/2606.03741.

B. Chen, X. Li, Y. Liang, Z. Shi, and Z. Song. Bypassing the exponential dependency: Looped transformers efficiently learn in-context by multi-step gradient descent. In *Proceedings of The 28th International Conference on Artificial Intelligence and Statistics*, volume 258 of *Proceedings of Machine Learning Research*, pages 4447–4455. PMLR, 2025. URL https://proceedings.mlr.press/v258/chen25i.html.

H.-H. Chen. Thinking deeper, not longer: Depth-recurrent transformers for compositional generalization, 2026.

L. Chen, J. Li, C. Liang, N. Lao, and Q. Liu. Training-free looped transformers, 2026a.

W. Chen, T. Li, W. Huang, Y. Yin, L. Shang, and C. Qin. LoopMoE: Unifying iterative computation with mixture-of-experts for language modeling. *arXiv preprint arXiv:2606.04438*, 2026b. URL https://arxiv.org/abs/2606.04438.

Y. Chen, Z. Yang, Z. Liu, C. Lee, P. Xu, M. Shoeybi, B. Catanzaro, and W. Ping. AceReason-Nemotron: Advancing math and code reasoning through reinforcement learning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026c. URL https://openreview.net/forum?id=EgArbnS0BA.

Z. Cheng, S. Hao, T. Liu, F. Zhou, Y. Xie, F. Yao, Y. Bian, Y. Zhuang, N. Dey, Y. Zha, Y. Gu, K. Zhou, Y. Wang, Y. Li, R. Fan, J. She, C. Gao, A. Saparov, H. Li, T. W. Killian, M. Yurochkin, Z. Liu, E. P. Xing, and Z. Hu. Revisiting reinforcement learning for llm reasoning from a cross-domain perspective. *arXiv preprint arXiv:2506.14965*, 2025. doi: 10.48550/arXiv.2506.14965.

F. Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. URL https://arxiv.org/abs/1911.01547.

F. Chollet, M. Knoop, G. Kamradt, B. Landers, and H. Pinkard. ARC-AGI-2: A new challenge for frontier ai reasoning systems, 2025. URL https://arxiv.org/abs/2505.11831.

F. Chollet, M. Knoop, G. Kamradt, and B. Landers. ARC prize 2025: Technical report, 2026. URL https://arxiv.org/abs/2601.10904.

J. R. Chowdhury and C. Caragea. Investigating recurrent transformers with dynamic halt, 2024. URL https://arxiv.org/abs/2402.00976.

C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300. URL https://aclanthology.org/N19-1300/.

P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. URL https://arxiv.org/abs/1803.05457.

R. Csordás, K. Irie, J. Schmidhuber, C. Potts, and C. D. Manning. MoEUT: Mixture-of-experts universal transformers. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL https://openreview.net/forum?id=ZxVrkm7Bjl.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026. URL https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro.

M. Dehghani, S. Gouws, O. Vinyals, J. Uszkoreit, and L. Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=HyzdRiR9Y7.

C. Deng, Y. Zhang, R.-J. Zhu, Y. Xu, J. Liu, T. S. E. Ng, and H. Chen. LT2: Linear-time looped transformers, 2026.

N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, B. Zoph, L. Fedus, M. Bosma, Z. Zhou, T. Wang, Y. E. Wang, K. Webster, M. Pellat, K. Robinson, K. Meier-Hellstern, T. Duke, L. Dixon, K. Zhang, Q. V. Le, Y. Wu, Z. Chen, and C. Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5547–5569, 2022. URL https://arxiv.org/abs/2112.06905.

N. Dziri, X. Lu, M. Sclar, X. L. Li, L. Jian, B. Y. Lin, P. West, C. Bhagavatula, R. Le Bras, J. D. Hwang, et al. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, 2023.

M. Elbayad, J. Gu, E. Grave, and M. Auli. Depth-adaptive transformer. In *International Conference on Learning Representations*, 2020.

A. Fan, T. Lavril, E. Grave, A. Joulin, and S. Sukhbaatar. Addressing some limitations of transformers with feedback memory, 2020.

Y. Fan, Y. Du, K. Ramchandran, and K. Lee. Looped transformers for length generalization. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=2edigk8yoU.

Y. Fan, A. Svete, and K. Lee. Bridging the gap between latent and explicit reasoning with looped transformers, 2026. URL https://arxiv.org/abs/2606.31779.

W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://arxiv.org/abs/2101.03961.

J. Fein-Ashley and P. Rashidinejad. Solve the loop: Attractor models for language and reasoning, 2026. URL https://arxiv.org/abs/2605.12466.

M. Frey, B. Shomali, A. H. Bashir, D. Berghaus, J. Koehler, and M. Ali. Adaptive loops and memory in transformers: Think harder or know more? In *Workshop on Latent and Implicit Thinking, International Conference on Learning Representations*, 2026a. URL https://arxiv.org/abs/2603.08391.

M. Frey, B. Shomali, J. Koehler, and M. Ali. A dual-path architecture for scaling compute and capacity in LLMs. *arXiv preprint arXiv:2605.30202*, 2026b. URL https://arxiv.org/abs/2605.30202.

H. Fu, T. Guo, Z. Wang, H. Zhu, J. D. Lee, J. Jiao, S. Russell, and S. Mei. Discoloop: Looping discrete embeddings and continuous hidden states for multi-hop reasoning, 2026a. URL https://arxiv.org/abs/2607.00341.

R. Fu, Z. Yang, J. Zhang, J. Ma, H. Chen, Y. Li, and Y. Chang. Simply stabilizing the loop via fully looped transformer, 2026b. URL https://arxiv.org/abs/2605.18797.

T. Fu, Y. You, Z. Chen, G. Dai, H. Yang, and Y. Wang. Think-at-hard: Selective latent iterations to improve reasoning language models, 2025.

L. Gao, J. Tow, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. A framework for few-shot language model evaluation, 2024. URL https://github.com/EleutherAI/lm-evaluation-harness.

Y. Gao, C. Zheng, E. Xie, H. Shi, T. Hu, Y. Li, M. K. Ng, Z. Li, and Z. Liu. Algoformer: An efficient transformer framework with algorithmic structures, 2025a. URL https://arxiv.org/abs/2402.13572.

Z. Gao, L. Chen, Y. Xiao, H. Xing, R. Tao, H. Luo, J. Zhou, and B. Dai. Universal reasoning model, 2025b. URL https://arxiv.org/abs/2512.14693.

Z. Gao, H. Luo, L. Chen, J. K. Liu, R. Tao, J. Zhou, and B. Dai. What makes diffusion language models super data learners?, 2025c. URL https://arxiv.org/abs/2510.04071.

K. Gatmiry, N. Saunshi, S. J. Reddi, S. Jegelka, and S. Kumar. Can looped transformers learn to implement multi-step gradient descent for in-context learning? In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 15130–15152. PMLR, 2024a. URL https://proceedings.mlr.press/v235/gatmiry24b.html.

K. Gatmiry, N. Saunshi, S. J. Reddi, S. Jegelka, and S. Kumar. On the role of depth and looping for in-context learning with task diversity. *arXiv preprint arXiv:2410.21698*, 2024b. URL https://arxiv.org/abs/2410.21698.

R. Ge, Q. Liao, and T. Poggio. Hierarchical reasoning models: Perspectives and misconceptions, 2025. URL https://arxiv.org/abs/2510.00355.

J. Geiping, S. McLeish, N. Jain, J. Kirchenbauer, S. Singh, B. R. Bartoldson, B. Kailkhura, A. Bhatele, and T. Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025. doi: 10.48550/arXiv.2502.05171. URL https://arxiv.org/abs/2502.05171.

A. Giannou, S. Rajput, J.-y. Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos. Looped transformers as programmable computers. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 11398–11442. PMLR, 2023.

GLM-5 Team, A. Zeng, X. Lv, Z. Hou, et al. Glm-5: from vibe coding to agentic engineering, 2026. URL https://arxiv.org/abs/2602.15763.

X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In Y. W. Teh and M. Titterington, editors, *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, volume 9 of *Proceedings of Machine Learning Research*, pages 249–256. PMLR, 2010. URL https://proceedings.mlr.press/v9/glorot10a.html.

A. Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016. URL https://arxiv.org/abs/1603.08983.

T. Guo, H. Zhu, R. Zhang, J. Jiao, S. Mei, M. I. Jordan, and S. Russell. How do LLMs perform two-hop reasoning in context?, 2025. URL https://arxiv.org/abs/2502.13913.

S.-I. Han. Hierarchical vs. flat iteration in shared-weight transformers, 2026. URL https://arxiv.org/abs/2604.14442.

J. He, J. Liu, C. Y. Liu, R. Yan, C. Wang, P. Cheng, X. Zhang, F. Zhang, J. Xu, W. Shen, S. Li, L. Zeng, T. Wei, C. Cheng, B. An, Y. Liu, and Y. Zhou. Skywork open reasoner 1 technical report. *arXiv preprint arXiv:2505.22312*, 2025.

D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. d. L. Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. v. d. Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 30016–30030, 2022.

A. Huang, A. Li, A. Kong, et al. Step 3.5 Flash: Open frontier-level intelligence with 11B active parameters, 2026a. URL https://arxiv.org/abs/2602.10604.

B. Huang, Z. Geng, and J. Z. Kolter. Equilibrium reasoners: Learning attractors enables scalable reasoning. In *Forty-third International Conference on Machine Learning*, 2026b. URL https://openreview.net/forum?id=lh95PnOlpM.

D. Hutchins, I. Schlag, Y. Wu, and B. Neyshabur. Block-recurrent transformers. In *Advances in Neural Information Processing Systems*, 2022.

N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

G. Jawahar, B. Sagot, and D. Seddah. What does BERT learn about the structure of language? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3651–3657, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1356.

A. Jeddi, M. Ciccone, and B. Taati. Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=RzYXb5YWBs.

H. Jin, W. Wei, X. Wang, W. Zhang, and Y. Wu. Rethinking learning rate tuning in the era of large language models, 2023. URL https://arxiv.org/abs/2309.08859.

A. Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks, 2025. URL https://arxiv.org/abs/2510.04871.

L. Kaiser and I. Sutskever. Neural gpus learn algorithms. In *International Conference on Learning Representations*, 2016.

J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020. URL https://arxiv.org/abs/2001.08361.

Kimi Team, T. Bai, Y. Bai, Y. Bao, et al. Kimi k2.5: Visual agentic intelligence, 2026. URL https://arxiv.org/abs/2602.02276.

D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

H. Kohli, S. Parthasarathy, H. Sun, and Y. Yao. Loop, think, & generalize: Implicit reasoning in recurrent-depth transformers, 2026. URL https://arxiv.org/abs/2604.07822.

M. Komisarczyk, S. Mathur, M. Kraus, S. Natarajan, and K. Kersting. Recursive inference machines for neural reasoning, 2026. URL https://arxiv.org/abs/2603.05234.

V. A. Korthikanti, J. Casper, S. Lym, L. McAfee, M. Andersch, M. Shoeybi, and B. Catanzaro. Reducing activation recomputation in large transformer models. In D. Song, M. Carbin, and T. Chen, editors, *Proceedings of Machine Learning and Systems*, volume 5, pages 341–353. Curan, 2023. URL https://proceedings.mlsys.org/paper_files/paper/2023/file/80083951326cf5b35e5100260d64ed81-Paper-mlsys2023.pdf.

H.-Y. Kuo, E. M. Chayti, P. Reizinger, W. Brendel, and M. Jaggi. Stabilizing extrapolation in looped transformers via learned stochastic stopping, 2026. URL https://arxiv.org/abs/2606.29983.

D. Kwon and Y. Park. Cherry: Compressed hierarchical experts with recurrent representational yield, 2026. URL https://arxiv.org/abs/2606.31796.

A. Labovich. Stability and generalization in looped transformers, 2026.

V. A. Lakkapragada. Cosmicfish-hrm: Adaptive reasoning via hierarchical recurrent mechanisms in compact language models, 2026. URL https://arxiv.org/abs/2605.28919.

Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut. ALBERT: A lite BERT for self-supervised learning of language representations. In *International Conference on Learning Representations*, 2020. URL https://openreview.net/forum?id=H1eA7AEtvS.

R. Lee, J. Biloki, E. J. Hu, and J. May. Sparse layers are critical to scaling looped language models. *arXiv preprint arXiv:2605.09165*, 2026. URL https://arxiv.org/abs/2605.09165.

K. Li. Taco-verified. Hugging Face dataset, 2025. URL https://huggingface.co/datasets/likaixin/TACO-verified.

R. Li, J. Fu, B.-W. Zhang, T. Huang, Z. Sun, C. Lyu, G. Liu, Z. Jin, and G. Li. Taco: Topics in algorithmic code generation dataset. *arXiv preprint arXiv:2312.14852*, 2023a.

Y. Li, M. E. Ildiz, D. Papailiopoulos, and S. Oymak. Transformers as algorithms: Generalization and stability in in-context learning. In *International Conference on Machine Learning*, 2023b.

Z. Li, Y. Li, and T. Zhou. Skip a layer or loop it? learning program-of-layers in llms, 2026. URL https://arxiv.org/abs/2606.06574.

Z. Liu, Z. Yang, Y. Chen, C. Lee, M. Shoeybi, B. Catanzaro, and W. Ping. AceReason-Nemotron 1.1: Advancing math and code reasoning through SFT and RL synergy. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=IaEqjWXd1d.

I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL https://openreview.net/forum?id=Bkg6RiCqY7.

W. Lu, Y. Yang, K. Lee, Y. Li, and E. Liu. Latent chain-of-thought? decoding the depth-recurrent transformer, 2025.

K. Luo, Z. Sun, H. Wen, X. Shi, J. Cui, C. Dang, K. Lyu, and W. Chen. How learning rate decay wastes your best data in curriculum-based llm pretraining. *arXiv preprint arXiv:2511.18903*, 2025. URL https://arxiv.org/abs/2511.18903.

J. Mattern, S. Jaghouar, M. Basra, J. Straube, M. Di Ferrante, F. Gabriel, J. M. Ong, V. Weisser, and J. Hagemann. Synthetic-1: Two million collaboratively generated reasoning traces from deepseek-r1. Prime Intellect blog, 2025. URL https://www.primeintellect.ai/blog/synthetic-1-release.

R. K. McGovern. Test-time adaptation of tiny recursive models, 2025. URL https://arxiv.org/abs/2511.02886.

S. McLeish, A. Li, J. Kirchenbauer, D. S. Kalra, B. R. Bartoldson, B. Kailkhura, A. Schwarzschild, J. Geiping, T. Goldstein, and M. Goldblum. Teaching pretrained language models to think deeper with retrofitted recurrence, 2025.

A. Miele, Y. Qin, A. Carballo-Castro, J. Deschenaux, and P. Frossard. Fixed-point masked generative modeling, 2026. URL https://arxiv.org/abs/2605.31215.

T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language*

Processing, pages 2381–2391, Brussels, Belgium, Oct.-Nov. 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1260. URL https://aclanthology.org/D18-1260/.

MiniMax, A. Chen, A. Li, B. Zhou, et al. The minimax-m2 series: Mini activations unleashing max real-world intelligence, 2026. URL https://arxiv.org/abs/2605.26494.

A. Mohtashami, M. Pagliardini, and M. Jaggi. Cotformer: A chain-of-thought driven architecture with budget-adaptive computation cost at inference, 2025. URL https://arxiv.org/abs/2310.10845.

I. M. Moosa, S. Lohit, Y. Wang, M. Chatterjee, and W. Yin. Understanding dynamic compute allocation in recurrent transformers, 2026. URL https://arxiv.org/abs/2602.08864.

S. Movahedi, V. Milovanović, S. L. Feigin, A. Theus, T. Hofmann, V. Boeva, T. K. Rusch, and A. Orvieto. Fixed-point reasoners: Stable and adaptive deep looped transformers, 2026. URL https://arxiv.org/abs/2606.18206.

T. Q. Nguyen and J. Salazar. Transformers without tears: Improving the normalization of self-attention. In *Proceedings of the 16th International Conference on Spoken Language Translation*, Hong Kong, Nov. 2019. Association for Computational Linguistics. URL https://aclanthology.org/2019.iwslt-1.17/.

E. Nichani, A. Damian, and J. D. Lee. How transformers learn causal structure with gradient descent, 2024.

NVIDIA. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset, 2024. URL https://huggingface.co/datasets/nvidia/Nemotron-CC.

NVIDIA. Nemotron 3 Nano: Open, efficient Mixture-of-Experts hybrid Mamba-Transformer model for agentic reasoning, 2025. URL https://arxiv.org/abs/2512.20848.

NVIDIA. Nemotron 3 Super: Open, efficient Mixture-of-Experts hybrid Mamba-Transformer model for agentic reasoning, 2026a. URL https://arxiv.org/abs/2604.12374.

NVIDIA. Nemotron 3 Ultra: Open, efficient Mixture-of-Experts hybrid Mamba-Transformer model for agentic reasoning, 2026b. URL https://arxiv.org/abs/2606.15007.

F. Pappone, D. Crisostomi, and E. Rodolà. Two-scale latent dynamics for recurrent-depth transformers, 2025. URL https://arxiv.org/abs/2509.23314.

T. Park, Y. Lee, D. Kim, and H. Bae. Loopus: Recasting pretrained llms into looped latent refinement models, 2026. URL https://arxiv.org/abs/2605.11011.

H. Prairie, Z. Novack, T. Berg-Kirkpatrick, and D. Y. Fu. Parcae: Scaling laws for stable looped language models. *arXiv preprint arXiv:2604.12946*, 2026. URL https://arxiv.org/abs/2604.12946.

Qwen Team. Qwen3 technical report, 2025. URL https://qwenlm.github.io/blog/qwen3/.

S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, SC '20. IEEE Press, 2020. ISBN 9781728199986.

P. Rauba, C. Fanconi, and M. van der Schaar. Tiny autoregressive recursive models, 2026. URL https://arxiv.org/abs/2603.08082. ICLR 2026 Workshop RSI Spotlight.

Z. Ren and Z. Liu. Are your reasoning models reasoning or guessing? a mechanistic analysis of hierarchical reasoning models, 2026. URL https://arxiv.org/abs/2601.10679.

A. Roye-Azar, S. Vargas-Naranjo, D. Ghai, N. Balamurugan, and R. Amir. Tiny recursive models on ARC-AGI-1: Inductive biases, identity conditioning, and test-time compute, 2025. URL https://arxiv.org/abs/2512.11847.

K. Sakaguchi, R. Le Bras, C. Bhagavatula, and Y. Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8732–8740, 2020. doi: 10.1609/aaai.v34i05.6399. URL https://doi.org/10.1609/aaai.v34i05.6399.

G. Sapunov. Universal transformers need memory: Depth-state trade-offs in adaptive recursive reasoning, 2026. URL https://arxiv.org/abs/2604.21999.

N. Saunshi, N. Dikkala, Z. Li, S. Kumar, and S. J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=din0lGfZFd.

K. Schwethelm, D. Rueckert, and G. Kaissis. How much is one recurrence worth? iso-depth scaling laws for looped language models. *arXiv preprint arXiv:2604.21106*, 2026. URL https://arxiv.org/abs/2604.21106.

A. Sghaier, A. Parviz, and A. Jolicoeur-Martineau. Probabilistic tiny recursive model, 2026. URL https://arxiv.org/abs/2605.19943.

N. Shazeer and M. Stern. Adafactor: Adaptive learning rates with sublinear memory cost. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 4596–4604. PMLR, 10–15 Jul 2018. URL https://proceedings.mlr.press/v80/shazeer18a.html.

N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=B1ckMDqlg.

M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

W.-J. Shu, X. Qiu, R.-J. Zhu, H. H. Chen, Y. Liu, and H. Yang. LoopViT: Scaling visual ARC with looped transformers, 2026. URL https://arxiv.org/abs/2602.02156.

Q. Sun, M. Pickett, A. K. Nain, and L. Jones. Transformer layers as painters. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(24):25219–25227, 2025. doi: 10.1609/aaai.v39i24.34708.

S. Takase and S. Kiyono. Lessons on parameter sharing across layers in transformers. In *Proceedings of the Fourth Workshop on Simple and Efficient Natural Language Processing*, pages 78–90, Toronto, Canada, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.sustainlp-1.5. URL https://aclanthology.org/2023.sustainlp-1.5/.

S. Takase, S. Kiyono, S. Kobayashi, and J. Suzuki. Spike no more: Stabilizing the pre-training of large language models. In *Proceedings of the Conference on Language Modeling*, 2025. URL https://openreview.net/forum?id=52YBEzcI0l. COLM 2025.

A. Talmor, J. Herzig, N. Lourie, and J. Berant. CommonsenseQA: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4149–4158, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1421. URL https://aclanthology.org/N19-1421/.

S. Tan, M. Luo, J. Wong, C. Cai, X. Shi, W. Y. Tang, M. Roongta, T. Zhang, L. E. Li, R. A. Popa, and I. Stoica. Deepscaler: Effective rl scaling of reasoning models via iterative context lengthening. OpenReview, 2025. URL https://openreview.net/forum?id=I6GzDCne7U.

G. Tang, S. Jiang, H. Chang, N. Chen, Y. Li, H. Fan, J. Li, M. Liu, and B. Qin. Looprpt: Reinforcement pre-training for looped language models, 2026.

Team OLMo. OLMo 3, 2026. URL https://arxiv.org/abs/2512.13961.

I. Tenney, D. Das, and E. Pavlick. BERT rediscovers the classical NLP pipeline. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4593–4601, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1452.

S. Vahdati, A. Aioanei, H. Suresh, and J. Lehmann. The ARC of progress towards AGI: A living survey of abstraction and reasoning, 2026. URL https://arxiv.org/abs/2603.13372.

J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, 2023.

B. Wang, X. Yue, Y. Su, and H. Sun. Grokked transformers are implicit reasoners: A mechanistic journey to the edge of generalization. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2405.15071.

G. Wang, J. Li, Y. Sun, X. Chen, C. Liu, Y. Wu, M. Lu, S. Song, and Y. A. Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025. URL https://arxiv.org/abs/2506.21734.

G. Wang, C. Liu, C. Wang, C. Zhou, Y. Sun, Y. Wu, S. Zhen, L. Scimeca, and Y. A. Yadkori. Hrm-text: Efficient pretraining beyond scaling, 2026a. URL https://arxiv.org/abs/2605.20613.

S. Wang, B. Li, G. Zhang, W. Huang, S. Yan, and J. Li. On the residual scaling of looped transformers: Stability and transferability, 2026b. URL https://arxiv.org/abs/2606.18524.

K. Wen, Z. Li, J. Wang, D. Hall, P. Liang, and T. Ma. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective. *arXiv preprint arXiv:2410.05192*, 2024. URL https://arxiv.org/abs/2410.05192.

B. Wu, M. Chen, X. Luo, S. Yan, Q. Yu, F. Xia, T. Zhang, H. Zhan, Z. Zhong, X. Zhou, S. Qiao, and X. Bin. Parallel loop transformer for efficient test-time computation scaling, 2025. URL https://arxiv.org/abs/2510.24824.

L. Wu, C. Zhang, and Y. Cao. Looped transformers with layer normalization provably learn the power method, 2026. URL https://arxiv.org/abs/2606.00605.

Y. Wu, H. Wang, P. Zhao, Y. Zheng, Y. Wei, and L.-K. Huang. Mitigating catastrophic forgetting in online continual learning by modeling previous task interrelations via pareto optimization. In *Forty-first International Conference on Machine Learning*, 2024. URL https://openreview.net/forum?id=olbTrkWo1D.

Y. Xia, T. He, X. Tan, F. Tian, D. He, and T. Qin. Tied transformers: Neural machine translation with shared encoder and decoder. In *AAAI Conference on Artificial Intelligence*, 2019.

Y. Xia, W. Shen, Y. Wang, J. K. Liu, H. Sun, S. Wu, J. Hu, and X. Xu. Leetcodedataset: A temporal dataset for robust evaluation and efficient training of code llms. *arXiv preprint arXiv:2504.14655*, 2025.

S. M. Xie, A. Raghunathan, P. Liang, and T. Ma. An explanation of in-context learning as implicit bayesian inference. In *International Conference on Learning Representations*, 2022.

K. Xu and I. Sato. To CoT or to loop? a formal comparison between chain-of-thought and looped transformers, 2025a. URL https://arxiv.org/abs/2505.19245.

K. Xu and I. Sato. On expressive power of looped transformers: Theoretical analysis and enhancement via timestep encoding. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 69613–69646. PMLR, 2025b. URL https://proceedings.mlr.press/v267/xu25x.html.

L. Yang, K. Lee, R. D. Nowak, and D. Papailiopoulos. Looped transformers are better at learning learning algorithms. In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=HHbRxoDTxE.

K. Yano, S. Kiyono, S. Kobayashi, S. Takase, and J. Suzuki. Pre-training llm without learning rate decay enhances supervised fine-tuning. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2603.16127.

Y. Yao, Y. Du, D. Zhu, M. Hahn, and A. Koller. Language models can learn implicit multi-hop reasoning, but only if they have lots of training data. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 9684–9702, Suzhou, China, 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.emnlp-main.490. URL https://aclanthology.org/2025.emnlp-main.490/.

C. Yu, X. Shu, Y. Wang, Y. Zhang, H. Wu, Y. Wu, R. Long, Z. Chen, Y. Xu, W. Su, and B. Zheng. Spiralformer: Looped transformers can learn hierarchical dependencies via multi-resolution recursion, 2026. URL https://arxiv.org/abs/2602.11698.

Q. Yu, Z. Zhang, R. Zhu, Y. Yuan, X. Zuo, Y. Yue, W. Dai, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

A. Zeitoun, L. Torroba-Hennigen, and Y. Kim. Hyperloop transformers, 2026. URL https://arxiv.org/abs/2604.21254.

R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472. URL https://aclanthology.org/P19-1472/.

R. Zhang, S. Frei, and P. L. Bartlett. Trained transformers learn linear models in-context, 2023.

X. Zhang, H. Wu, G. He, J. Shen, B. Lyu, and Z. Zhu. Modr: Mixture-of-depth-recurrent transformers for test-time reasoning. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=9Pba4rcQbE.

C. Zheng, S. Liu, M. Li, X.-H. Chen, B. Yu, C. Gao, K. Dang, Y. Liu, R. Men, A. Yang, J. Zhou, and J. Lin. Group sequence policy optimization, 2025. URL https://arxiv.org/abs/2507.18071.

Y. Zheng, X. Wang, L. Ren, and C. Wei. Chaingpt: Dual-reasoning model with recurrent depth and multi-rank state updates. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=kdZbxizwGK.

R.-J. Zhu, Z. Wang, K. Hua, T. Zhang, Z. Li, H. Que, B. Wei, Z. Wen, F. Yin, H. Xing, L. Li, J. Shi, K. Ma, S. Li, T. Kergan, A. Smith, X. Qu, M. Hui, B. Wu, Q. Min, H. Huang, X. Zhou, W. Ye, J. Liu, J. Yang, Y. Shi, C. Lin, E. Zhao, T. Cai, G. Zhang, W. Huang, Y. Bengio, and J. Eshraghian. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025. doi: 10.48550/arXiv.2510.25741. URL https://arxiv.org/abs/2510.25741.

E. Çakar, K. A. Raghu, and L. Zheng. Laser: Low-rank activation svd for efficient recursion, 2026. URL https://arxiv.org/abs/2604.17224.

A 架构细节

表 4 Loopie-20B-A2B 与 Loopie-6B-A0.6B 的架构细节

	Loopie-20B-A2B	Loopie-6B-A0.6B
Layer-loop times	2	2
QK layer norm	Enabled	Enabled
Num layers	27	18
Hidden size	2304	1536
MoE hidden size	832	576
Attention type	GQA	GQA
Attention heads	72	48
Attention groups	36	24
Position embedding type	RoPE	RoPE
Rotary base	10000	10000
RMSNorm eps	1e-6	1e-6
MoE router topk	8	8
Num experts	128	128
MLP type	SwiGLU	SwiGLU
Tokenizer	Qwen3	Qwen3
Vocabulary size	151936	151936

B 预训练细节

表 5 Loopie-20B-A2B 与 Loopie-6B-A0.6B 的预训练第一阶段细节

	Loopie-20B-A2B	Loopie-6B-A0.6B
Initialization std	$1/\sqrt{2.5 * 2304}$	$1/\sqrt{2.5 * 1536}$
LM head Initialization std	$1/\sqrt{2304}$	$1/\sqrt{1536}$
嵌入初始化标准差	$1/\sqrt{2304}$	$1/\sqrt{1536}$
专家混合辅助损失系数 0.01 0.01		
学习率	3×10^{-4}	5×10^{-4}
学习率调度 预热后稳定 预热后稳定		
预热步数 6000 2000		
全局批大小 1024 1024		
序列长度 8192 8192		
优化器 AdamW AdamW		
β_1	0.9	0.9
β_2	0.95	0.95
Weight decay	0.1	0.1
Clip gradient	1.0	1.0
Adam epsilon	1×10^{-15}	1×10^{-15}

表 6 预训练第二阶段 Loopie-20B-A2B 与 Loopie-6B-A0.6B 的详细设置

	Loopie-20B-A2B	Loopie-6B-A0.6B
Learning rate	3×10^{-4}	5×10^{-4}
学习率调度 恒定 恒定		
专家混合辅助损失系数 0.01 0.01		
全局批大小 1024 1024		
序列长度 8192 8192		
优化器 AdamW AdamW		
β_1	0.9	0.9
β_2	0.95	0.95
Weight decay	0.1	0.1
Clip gradient	1.0	1.0
Adam epsilon	1×10^{-15}	1×10^{-15}

C 缩放阶梯详情

Rung	Model	Total Params	Active Params	L	D	D_{MoE}	Heads	Head Dim	N	Width/ Depth	Tokens
1	Vanilla	1.34B	0.15B	27	640	192	20	32	1	23.70	150B
	Loopie	1.08B	0.11B	15	704	256	22	32	2	23.47	150B
2	Vanilla	2.37B	0.25B	30	768	256	24	32	1	25.60	250B
	Loopie	1.81B	0.18B	17	832	320	26	32	2	24.47	250B
3	Vanilla	4.76B	0.51B	36	1280	320	32	40	1	35.56	500B
	Loopie	3.78B	0.41B	19	1280	384	32	40	2	33.68	500B
4	Vanilla	9.14B	1.00B	46	1280	384	40	32	1	27.83	500B
	Loopie	6.36B	0.68B	25	1408	448	44	32	2	28.16	500B

表 7 Loopie 缩放阶梯的架构详情。每个 Loopie 模型与一个非循环的普通 MoE 基线配对，在匹配的预训练墙钟时间下进行。此处 D 表示隐藏维度，D_{MoE} 表示每个专家的隐藏大小，L 表示存储的 Transformer/MoE 层数，Heads 表示注意力头数，Head Dim 表示每个注意力头的维度，N 表示循环层循环步数，Width/Depth 表示 $D/(LN)$。

D 监督预训练细节

表 8 监督预训练中 Loopie-20B-A2B 和 Loopie-6B-A0.6B 的训练超参数。

	Loopie-20B-A2B	Loopie-6B-A0.6B
Global batch size	1024	1024
Sequence length	131072	131072
Learning rate	1×10^{-5}	1×10^{-5}
学习率调度 预热后稳定 预热后稳定		
预热步数 500 500		
专家混合辅助损失系数 0.01 0.01		
优化器 AdamW AdamW		
β_1	0.9	0.9
β_2	0.95	0.95
Weight decay	0.1	0.1
Clip gradient	1.0	1.0
Adam epsilon	1×10^{-15}	1×10^{-15}